

CUHK-X 方法论教学详解 —— 每个用过的/在用的方法,讲透"思想 → 实现 → 实例 → 边界 → 面试"

配套阅读: doc/PROJECT_WALKTHROUGH.md (全局复盘/从零搭建/试错史)+ doc/PLAN.md (权威规划)。

本文目的:把项目里**用过或正在用的**每一个方法论,拆成五件事讲清楚——

① **思想**(为什么这么做,背后的直觉);② **具体实现**(对应到工作区真实代码,给出可复述的伪码/要点);③ **项目实例**(本项目里的具体数字/事件);④ **适用边界**(什么时候不要用);⑤ **面试一句话**(怎么在 30 秒内讲出亮点)。

阅读建议:第一遍通读"①思想 + ③实例"建立直觉;第二遍再抠"②实现"的代码;面试前只看"⑤面试一句话"做自查。

方法族谱(先把地图铺开)

- └─ 任务定义层: RGB-Free / 跨被试 / $\leq 100\text{MB}$ 三重约束(决定一切取舍)
 - |
- └─ 数据工程
 - | └─ M8 跨被试评估协议(subject-split / 防泄漏) ← 尺子
 - | └─ M9 类平衡采样(WeightedRandomSampler) ← 治长尾 29.5
 - | └─ M10 模态索引构建(discovery 模态 / 帧号配对 / 兜底) ← 6 模态怎么对齐
 - | └─ M17 输入归一化(Kinetics vs ImageNet) ← 域匹配
 - |
- └─ 单模态建模(每模态一条线,先独立达标)
 - | └─ 视觉主线 M3/M4 R(2+1)D-34 + IG-65M(分解卷积+预训练迁移 M14)
 - | └─ 检测定位 M6/M7 YOLO + 固定窗口裁剪(检测-识别解耦)
 - | └─ 训练配方 M11 增强 / M12 label smoothing / M13 调度器 / M15 微调lr
 - | └─ 骨架线 M18-M20 ST-GCN → MotionBERT
 - | └─ Thermal M14 同构迁移 + M17 归一化(同一条视觉配方)
 - |
- └─ 约束工程
 - | └─ M21-M23 量化与 bit-packing(100MB 数学账)
 - | └─ M28 TTA(flip / time)
 - | └─ M25 集成(logits/prob 平均 / SWA / seed)
 - |
- └─ 多模态融合
 - | └─ M24 融合层次全景(早/特征/晚/决策)
 - | └─ M26 oracle / 互补性分析(先算天花板再投入)
 - | └─ M27 门控/类感知/动态加权(试过并关闭)
 - |
- └─ 半监督与表征(试过并关闭,但要能讲)
 - | └─ M29 对比学习 InfoNCE / CLIP
 - | └─ M30 伪标签自训练 Co-training
 - | └─ M31 知识蒸馏
 - | └─ M32 外部数据迁移(NTU-60)
 - |
- └─ 损失与决策
 - | └─ M33 CE / label smoothing / ArcFace / CenterLoss
 - | └─ M34 辅助回归头 MPJPE(auxpose)
 - | └─ M35 v3 bounded-prior 类偏置修正(在用,推理侧)
 - | └─ M36-M38 实验决策框架(fold0 判据 / 不烧 LB / 小数据总纲)

阅读本文最重要的心理建设:这个项目 0.73 的成绩 **不是靠任何一个"炫技方法"**,而是靠把上面这些**成熟方法系统性**地**用对 + 用对地方 + 用对配方**。所以每一个方法的教学都要带一个问题:"它在这套约束下,到底该不该用、该怎么用?"

前置知识与名词速查表(小白必读,看不懂时先看这里)

给"了解不深"的读者:本表把全文会用到的所有名词用一句大白话钉死。
用法:先全扫一遍有个印象;读某个 M 节卡住时,回来看对应的类目;遇到 Mxx 是本文的方法编号。

A. 深度学习最最基础(这一栏都不懂,先补这个)

名词	大白话
神经网络 / 参数	一个由大量"可调旋钮(参数)"组成的函数,图像喂进去,出来一个预测
训练	根据"预测 vs 答案"的差距(损失),用梯度下降把旋钮一点点调准
损失(loss)	预测比答案差多少的度量,越小越好
前向 / 反向传播	正向算出预测;反向把"差距"分摊给每个参数,告诉它往哪拧
epoch(轮)	把整个训练集完整过一遍模型
batch(批)	一次一起喂的样本数(如 32 个);训练按批进行
过拟合 / 欠拟合	过拟合=把训练集背下来、换数据就失灵;欠拟合=连训练集都没学明白
正则化	一切"故意不让模型背太熟"的手段(dropout / 增强 / 权重衰减...)
训练集 / 验证集(val) / 测试集	调参用 / 选模型、看能否泛化用 / 最终真实成绩用
交叉验证 / 折(fold)	把数据切成 K 份轮着当验证集做 K 次实验取平均;每份叫一折
accuracy(准确率)	猜对的样本比例(0.73 = 73%)
消融实验(ablation)	只改一个变量、其余全不变,看这个变量的贡献
基线(baseline)	最朴素的对照方案,先跑通它,再在其上叠加改进

B. 架构与算子

名词	大白话
卷积(CNN) / 卷积核	用一个小窗口扫遍整张图提取"局部模式"的操作;卷积核=这个小窗口
通道(channel)	特征图里的一"页"(RGB 图=3 页;深层网络几十上百页)
pool(池化) / stride(步长)	缩小尺寸的操作 / 扫描时一次挪几格

名词	大白话
ReLU / BN / Dropout	激活函数(把负数归零)/ 每层标准化(稳训练)/ 训练时随机丢一部分神经元(正则)
backbone(主干) / stem / head	网络主体(负责提特征)/ 最前面的入门口(处理原始输入)/ 最后的分类输出层
logits(原始分数)	最后一层对每个类输出的"支持票数",任意大小、可正可负
softmax	把 logits 变成长"加起来=1 的概率分布"的函数
one-hot 标签	答案的表示:正确类=1,其余=0
交叉熵(CrossEntropy)	度量"模型概率分布 vs 答案"差距的损失
transformer / attention	现代架构:让每个位置"全局看一遍所有位置、按重要度加权混合信息"
embedding(特征)	网络中间层产出的向量化表示(一段数字,代表"这个输入是什么")
图卷积(GCN)	在"图(节点+边)"上做信息聚合的卷积,骨架类任务用

C. 数据与任务

名词	大白话
模态(modality)	一种传感器数据(红外是一种模态,热像是一种模态)
clip(片段)	一段"一次动作"的数据(含若干帧图像/骨架)
帧(frame) / fps	一张画面 / 每秒帧数
RGB-Free	任务不提供彩色 RGB 图(隐私场景)
跨被试(cross-subject)	测试的人训练时从没见过(本任务最难的点)
被试(subject)	参与录制的人(user1、user2...)
长尾 / 类别不平衡	有的类几百个样本、有的类只有十几个
伪彩色(pseudocolor)	用假颜色编码一个物理量(如深度、温度)的图,不是真实照片
归一化(normalize)	把数值统一到某个范围(如匹配预训练见过的范围)
增强(augmentation)	训练时对输入做合理变形(翻转/亮度/缩放...)——等价于白送数据
对齐(alignment)	两种数据在时间轴/坐标系上能不能一一对上
早融合 / 晚融合	输入端拼通道 / 各自建模后在输出端合并

D. 训练与正则

名词	大白话
预训练 / 微调 / 迁移学习	先在超大数据集学会通用能力,再用小数据精调
学习率(lr)	每步参数"拧多大":太大冲过头、太小学不动
调度器(scheduler)	训练过程中 lr 按什么规律变化(降几次、降到哪)
采样器 / WeightedRandomSampler	决定每个 batch "抽哪些样本";带权重的=按权重抽(用于平衡类别)
label smoothing(标签平滑)	把 one-hot 揉软一点,逼模型别过度自信
早停(early stopping)	验证集不再涨就提前停,防过拟合
TTA(测试时增强)	推理时对输入做几种合理变形、把输出平均,白赚稳健
seed(种子)	随机数起点;同 seed 结果可复现
集成(ensemble)	多个模型的输出合并(平均),一般更稳
蒸馏 / 软标签	用强模型的概率当"软答案"去教弱模型
对比学习 / 正负样本对	拉近"同一语义的两种视角",推开"不同样本";InfoNCE 是它的损失
度量学习 / ArcFace	额外要求"同类聚拢、异类分开"的损失家族

E. 评估 / 竞赛 / 融合分析

名词	大白话
LB(排行榜)	竞赛测试集上的公开成绩,最终裁判
泄漏(data leakage)	不该给模型看的"答案线索"混进训练(如同一人进了验证集)
身份捷径	模型靠"认人/认房间"而非"认动作"得分
oracle(神谕)	假设每个样本都"完美选最佳"时融合能到的理论上限
rescue / corruption	融合分析:副模态救回主模态多少错 / 引入多少新错
宏平均 / 微平均	每类先算准确率再平均 / 所有样本一起算

F. 部署 / 工程 / 环境

名词	大白话
量化(quantization) / bit	用更少的位存参数(8bit/5bit/4bit);bit 越低体积越小、误差越大
fp32 / int8	参数用 32 位浮点(4 字节/个)/ 用 8 位整数(1 字节/个)存
per-channel 量化	量化时每个输出通道单独给一个缩放尺度(损失更小)
bit-packing(位打包)	把 5bit/6bit 参数按位塞进字节流,进一步省空间
SWA(权重平均)	把多个模型的"参数"直接平均(本项目证明会翻车)
概率平均 vs logits 平均	把多个模型输出的概率(或原始分数)平均之后再判类
SLURM / sbatch	服务器任务调度系统 / 提交一个训练任务的脚本
conda 环境	隔离的 Python 包环境
HF 镜像(hf-mirror)	HuggingFace 下载的加速代理,网络差时用它
CRLF / LF	两种换行符;Windows 写的脚本到 Linux 会报错

G. 外部数据集 / 论文 / 工具(知道是"干这个的"即可)

名词	大白话
Kinetics-400 / IG-65M	视频理解数据集 / 它的巨型预训练版(6600 万视频), 动作识别界的"老司机"
H3.6M / AMASS	3D 人体运动数据集(MotionBERT 预训练的知识来源)
NTU-60	骨架动作识别公开数据集(40 人、约 5.7 万样本, 本文用它对骨架外部增强)
H3.6M-17 / COCO-17	两种"17 关节"骨架拓扑,连接顺序不同,不可混用
MPJPE	骨架 3D 预测 vs 真值的平均关节距离误差(越小越好)
MMPose	开源姿态估计工具(从图像里提取人体骨架)
LOSO	Leave-One-Subject-Out,最严格的跨被试评估
ST-GCN / CTR-GCN / PoseConv3D	骨架动作识别的经典及变体(图卷积方向)
VideoMAE / SlowFast / X3D / C3D	其他视频理解模型(本项目试过/对比过)
Swin Transformer	视觉 transformer 的重要版本(分窗口注意力)

名词	大白话
GBDT(XGBoost)	传统机器学习(树模型集成),常用于把小特征喂给它的次级分类器
Otsu(大津法)	自动找一个阈值把灰度图分成前景/背景

顺带说明:本文出现 `Mxx` = 方法编号(第二部分到第十部分); `fold0` = 第 0 折验证结果; `LB` = 排行榜成绩; 3 折 = 做了 3 次交叉验证取平均。

第一部分:时序建模(动作识别的"一等公民")

项目宪法 **P1 运动优先**:动作 = 身体随时间的运动模式。这一部分讲清楚"为什么要建模时间"以及"模型层面怎么建模时间"。

M1. 为什么要"运动优先":从图像分类到视频分类

① 思想

动作识别和图像分类的本质区别:图像分类学的是"静态外观"(这只猫长什么样),动作识别学的是"随时间变化的模式"(这个人**怎么**完成喝水)。单帧里信息高度歧义——"手在嘴边的单帧"可能是喝水也可能是打电话;**时间维度把歧义消掉**。如果只用单帧,模型很容易走捷径去记"某个人的长相/某个房间的样子",而不是"动作本身"。

② 具体实现(本项目)

这是"思想层"的方法,体现为三条硬决策:

- 1. 输入是 **16 帧均匀采样的视频片段**(`DepthIRVideoDataset._uniform_indices`),不是单帧;
- 2. 主干必须是**视频模型**(R2+1D / TSM / 3D CNN),不是图像 CNN;
- 3. 数据增强、归一化全部作用于"整个片段"(`per-clip` 一致的增强参数,保证帧间一致性)。

③ 项目实例

- E0 基线用 TSM-ResNet18(2D 底子 + 时序移位)只有 0.3615;换 R2+1D(真分解卷积)到 0.4618。
- 对比实验进一步说明问题:IR Otsu 只抹背景、32 帧重复采样都对**运动信息**有害(0.27 / 0.33),证明"时序信息是命根子,动它要非常小心"。

④ 适用边界

纯静态任务(烟雾识别、场景分类)不需要强的时序建模;动作高度依赖速度/方向的才必须。任务的"时间粒度"决定帧数:短促动作要高频、长程动作要覆盖全程。

⑤ 面试一句话

"动作识别和图像分类的最大区别是必须建模时间——所以我们第一条宪法是'运动优先',所有模态的选择、裁剪、

增强都以'保住运动信息'为最高优先级。"

M2. TSM——"零参数的时序建模"与它在本项目的位置

① 思想

想让一个 2D CNN"看见"时间,但又不想引入 3D 卷积的参数爆炸?TSM(时序移位,Temporal Shift)的思路是:**在通道维度上把帧序列"错位平移"**——把某些通道的激活值"送"给相邻帧。这样 2D 卷积在卷某个空间位置时,输入里已经混入了前后帧的信息,等价于"廉价版的 3D 卷积"。**零新增参数、几乎零计算量**,是 2D 方案的性价比之王。

 **小白补课(先把"通道"讲明白)**:深度网络里,一张图输入后每层的输出不是一张图,而是**几十上百张"小特征图"叠在一起**,每一张叫一个**通道(channel)**。想象成一本"分页笔记本":输入 RGB 图 = 3 页(红、绿、蓝);越往深处页数越多(比如 64 页),每页记录一种模式("这里有竖边"、"这里有个人".....)。TSM 干的事 = 把这本笔记本里的**某些页整页往时间轴挪一帧**,让后面的卷积在扫同一位置时,同时读到"这一帧的这页"和"上一帧/下一帧的同页"——时间信息就被"混"进来了。"通道维度"就是笔记本的"页"这一维。

② 具体实现(本项目 src/model.py 的 TemporalShift)

输入 [B*T, C, H, W] (把批和帧并在一起),按 n_segment=16 重新排成 [nb, T, C, H, W],再把通道切成三段:

```
fold = c // self.fold_div                # 每段通道数(默认 1/4)
out[:, :-1, :fold] = x[:, 1:, :fold]    # 前 1/4 通道 ← 后一帧的这部分
out[:, 1:, fold:2*fold] = x[:, :-1, fold:2*fold] # 中间 1/4 通道 ← 前一帧这部分
out[:, :, 2*fold:] = x[:, :, 2*fold:]    # 后一半通道不动(自身)
```

一半通道"向前看一帧"、一半"保持自己",这样后面的 conv 天然同时看到 t 和 t±1 的特征。

骨架对应(TSMResNet18):stem → layer1/2/3 → **TSM 插在 layer3 之后再进 layer4** → 全局池化 → fc,输出 [B,T,40] 后**沿时间平均**得到 [B,40] (TSM 惯例的帧级 logits 平均)。

③ 项目实例

TSM-ResNet18 是 **E0 基线**(0.3615)。它性能低于 R2+1D(+10%),但它便宜、好调,让团队在**改主干之前先把数据管线(4ch 拼接/裁剪/类平衡)调通**——作为"探路车"使用。当主干升级为 R2+1D 后,TSM 就退出主线。

④ 适用边界

TSM 的短板:移位是"硬性的",丢失了长距离时序依赖,对速度敏感动作建模粗糙;它终究是"2D 的补丁"。帧率极低(每帧信息量大)或要求精细运动建模时,不如真 3D。

⑤ 面试一句话

"TSM 用一个零参数的通道移位把时间信息注入 2D 网络——我们用它做 E0 基线把数据管线打通,但消融证明它对时序的运动建模不够,主干升级到 R(2+1)D 后立涨 10 个点。"

M3. 3D 卷积与 3D CNN 的思想基础

① 思想

把卷积核从 2D (k, k) 扩展成 3D (t, k, k) ——同时扫空间和时间。c3d 是经典,ResNet3D(torchvision 的 video/resnet , R2Plus1dStem + BasicBlock + Conv3D)成了视频理解的事实标准骨架。3D CNN 信息最完整,但代价是**参数量大、优化难**,且对"小数据 + 从头训练"极不友好。

② 具体实现(本项目 src/model.py)

- 主干用 torchvision 的
`VideoResNet(block=BasicBlock, conv_makers=[...], layers=[3,4,6,3], stem=R2Plus1dStem, num_classes=400)`
搭 R2Plus1D34; R2Plus1D18 直接用 `torchvision.models.video.r2plus1d_18`。
- 输入 [B,T,C,H,W] $\rightarrow$ `permute(0,2,1,3,4)` 变 [B,C,T,H,W] (PyTorch 3D 期望的 NCDHW)。
- 分类头: `encoder.fc` 摘掉换 `Identity`, `head = Dropout(0.3) + Linear(feats, 40)`。

③ 项目实例

3D 底座直接用现成的预训练 **Kinetics-400 / IG-65M** 权重,只需要自己改两处——stem 从 3ch 变 4ch(M14)、把最后的分类头从 400 类换成 40 类。

④ 适用边界

纯 3D CNN 在小数据上容易过拟合;预算紧张时 2D+TSM 更划算。**"用什么时序建模器"要由消融决定,不是由信仰决定。**

⑤ 面试一句话

"3D 卷积让网络在同一套参数里同时卷空间和时间,是最完整的信息建模,但有参数与优化代价——我们的选择是它的'分解版' R(2+1)D,用更少的参数拿到更强的时序建模。"

M4. R(2+1)D——分解卷积,本项目主干的灵魂

① 思想

一句话:把 3D 卷积分解成一个纯空间卷积($1 \times 3 \times 3$) + 一个纯时间卷积($3 \times 1 \times 1$),中间夹 ReLU。

为什么更好(论文 A Closer Look at Spatiotemporal Convolutions for Action Recognition ,ECCV 2018 的核心论据):

1. **参数更少**:分解后两个子卷积的参数和比原 3D 卷积小,同等表达力下更省;
2. **优化更好**:中间多了一道 ReLU,网络被迫"先学空间、再在空间特征上学时间",缓解了深 3D 网络的优化困难;
3. **正则化**:强行解耦时空,天然防止过度耦合的空间-时间模式,对小数据更友好——这正是我们 2891 个样本最需要的。

 **小白补课(预训练/微调一句话)**:从零训练 = 招一个零基础的实习生从头教;预训练 = 先让它是看几千万条视频学会"什么是边缘、什么是动"这些通用能力,再来我们的 2891 个样本上微调(用很小的步长继续学)——就像

招了个十年老手,只需教他我们公司的 40 个动作。详细版在 **M14**。**预训练 ≠ 免费**:微调的学习率必须小(M15), 否则会把老手"洗脑"回实习生状态。

② 具体实现(本项目 `src/model.py` 的 `_Conv2Plus1D`)

```
midplanes = (in * out * 3*3*3) // (in*3*3 + 3*out) # 中间通道数(老版自算)
nn.Sequential(
    nn.Conv3d(in, midplanes, kernel_size=(1,3,3), stride=(1,s,s), padding=(0,p,p)), # 纯空间
    nn.BatchNorm3d(midplanes), nn.ReLU(inplace=True),
    nn.Conv3d(midplanes, out, kernel_size=(3,1,1), stride=(s,1,1), padding=(p,0,0)), # 纯时间
)
```

⚠ **midplanes 公式是本项目踩过一个真实坑**:当前 torchvision 的 `Conv2Plus1D` 由外部统一传 `midplanes`,而老版 IG-65M 权重是**每个 block 自己算 midplanes**——两者在 `inplanes#planes` 的 block(如 `layer2.0`)上尺寸对不上(288 vs 230),导致 `load_state_dict` 报错。修复 = 自定义 `_Conv2Plus1D` 复刻老版公式。这类**"权重与代码版本不匹配"**的坑,是工程里最常见、也最能体现排查能力的事。

③ 项目实例

- R2+1D-18 0.4618(+10% vs TSM) → 升级 R2+1D-34(63M,IG-65M)+ 固定 crop + 帧差 → **0.5988(3 折)**,之后再靠 full data + 双 seed 推到 0.71144。
- 失败反例(阶段 D):R2+1D-34 + IG-65M 用 `lr=1e-3` 只有 0.4252 —— **架构对,超参错,照样崩**。这提醒:"主干强"不等于"结果强",微调 `lr`(M15)才是关键。

④ 适用边界

R(2+1)D 不是万能:对帧率极低/片段极短的场景,纯时间卷积可能过拟合;大容量 + 大数据的场景下,非分解的 X3D/SlowFast 家族往往更强。对"小数据 + 中小模型"赛道,R(2+1)D 是黄金选择。

⑤ 面试一句话

"R(2+1)D 把 3D 卷积拆成'空间卷积+时间卷积',中间加 ReLU——参数更省、优化更好、天然的正则化,特别适合我们这种 2891 个样本的小数据动作识别,消融立涨 10 个点。我们还修了一个老版权重 `midplanes` 公式和 torchvision 不一致的兼容坑。"

M5. 时间采样:为什么是"16 帧均匀采样",而不是"每帧/前 16 帧"

① 思想

模型要看到"动作这一段",但 clip 长度不一(Depth/IR 平均 29 帧、Thermal 70+ 帧)。**"取哪 16 帧"**决定了模型看到的时间结构:

- 逐帧全用:算力爆炸,且相邻帧高度冗余;
- 只取前 16 帧:后期动作全看不到(长动作会被截断);

- **均匀采样(endpoint-uniform)**:把整段时间"压"成 16 帧,保全过程结构——短 clip 会重复、长 clip 会抽稀,但"从开始到结束"的时间结构不丢。

② 具体实现(本项目 `src/dataset.py._uniform_indices`)

```
if n <= self.num_frames:                # 帧不够:linSPACE 会重复采样补满 16
    return np.linspace(0, n-1, self.num_frames).round().astype(int)
return np.linspace(0, n-1, self.num_frames).round().astype(int)    # 够就均匀抽 16
```

(注意两分支看似一样,实际 `linSPACE` 本身就是均匀的;缺帧 clip 由外层 `common` 交集 + `_load` 补零兜底。)

③ 项目实例(一个作废的消融)

试过 **32 帧**:`val` 0.3305,反而比 16 帧差。(消融 = 只改"帧数"这一个变量、其余全不动,专门看它自己的影响)根因:clip 平均只有 ~29 帧,32 帧意味着大量"重复帧/近邻帧",**重复帧不给新信息,还稀释了均匀采样的结构**,同时对显存、训练时长是纯负担。**结论:帧数是"够覆盖结构就行",不是越多越好。**

④ 适用边界

帧数选择要 $\leq$ clip 平均长度(否则靠重复帧注水);若动作集中在片段局部(短促动作),均匀采样反而不利,要考虑"关键帧采样 / 时间 TTA"(见 M28)。

⑤ 面试一句话

"我们用 `endpoint-uniform` 把任意长度的 clip 压成 16 帧,保全过程时间结构;消融显示 32 帧反而变差,因为 clip 平均只有 29 帧、重复帧无新信息——帧数要贴着任务的时间粒度选。"

第二部分:检测与裁剪(项目两条宪法之一的落地)

M6. One-Stage 目标检测 YOLO——"定位人在哪"

① 思想

检测要回答"画面里有没有人、人在哪"。YOLO(You Only Look Once)把检测做成一次端到端的回归:把图像划分为网格,每个格子直接回归"框 + 置信度 + 类别"(one-stage),不需要 Two-Stage 的"先提候选框再分类"。极快、简化,足够在每帧上跑,是小模型赛道的检测标配。**注意我们只用到它的"人检测"能力(classes=[0]),不训练、不微调,纯用 YOLO11n 预训练权重——因为检测只负责"人在哪",不需要懂动作。**

 **小白补课(目标检测在干什么)**:分类回答"整张图属于哪类";检测回答"画面里哪里有物体、它占多大一个框"(输出若干个 类别+左上角+右下角)。有两条路线:**Two-Stage**(如 Faster R-CNN)先筛候选框再逐个细分类,慢但准;**One-Stage**(YOLO)把图切成网格、每格直接一次回归出框,快。如今 YOLO 又快又够准,成了主流;而我们只让它找"人"这一类(classes=[0]),别的不管。

② 具体实现(本项目 `src/detect.py`)

```

results = model.predict(rgb, classes=[0], conf=0.25, verbose=False, device=device)
if len(results[0].boxes):
    b = results[0].boxes.xyxy[results[0].boxes.conf.argmax()].tolist()
    # 归一化到 [0,1]: xyxy / (W, H)
    boxes.append([b[0]/W, b[1]/H, b[2]/W, b[3]/H])

```

- **在哪一帧上检测**: IR 帧(灰度人体剪影,对比度最高、受环境干扰最小)。训练集检测率 99%、测试 97.5%。也可 --modality thermal 在 Thermal 帧上检测(生成 bbox_thermal_train.json)。
- **探测多少帧**: 一个 clip 挑 **8 帧均匀帧**检测(不是逐帧),然后 union。
- **权重来源**: 本地没有则走 HF 镜像下载 yolo11n.pt (HF_ENDPOINT=https://hf-mirror.com)。

③ 项目实例

- **离线缓存**: 检测结果一次性算好存成 bbox_train.json / bbox_test.json (每 clip 一个归一化窗口),训练/推理直接读 cache,不在 DataLoader 里跑 YOLO(那是 40 倍浪费)。
- **消融: +YOLO 固定裁剪**(0.4618 → 0.5033, +4pt)。检测虽然"不懂动作",但把"动作发生的主体区域"圈出来,让视频模型专注看人。
- **检测失败自动兜底**: 检测不到就把整帧当窗口(0/1 全帧),绝不崩。

④ 适用边界

检测不是万能的:多目标场景(多人)要选主目标(我们取置信最高的那个);检测错位会引入噪声(见 M7 为什么用固定窗口);检测率低时(如部分 Thermal)要接受退化而不是强行用。

⑤ 面试一句话

"我们让 YOLO11n 只在 IR 上做'人检测'(99% 检测率),检测结果离线缓存成每 clip 一个固定窗口——因为定位和识别应当解耦:检测回答'人在哪',识别回答'在做什么'。"

M7. 检测-识别解耦 & 固定窗口裁剪——本项目最核心的"保运动"方法论

① 思想(为什么固定窗口,而不是逐帧裁剪)

动作识别最值钱的信息之一是**位移**:人从画面左边走到右边,是"走路"区别于"原地踏步"的关键。

- **逐帧裁剪** = 每一帧都重新贴住人 → 人在画面里永远在正中心 → **位移信息被完全抹平**,网络看到的是"一个人原地做动作",丢掉了最重要的空间运动线索;
- **多帧检测** → **取所有框的并集(union)** → **放大** → **全程用这同一个固定窗口裁所有帧** = 网络在一个固定坐标系里观察身体相对窗口的运动轨迹,位移变成可见特征。

② 具体实现(本项目 src/detect.py.window_from_boxes + dataset._load)

```
# ① 多帧框 union 成一个大框
x0, y0 = arr[:,0].min(), arr[:,1].min(); x1, y1 = arr[:,2].max(), arr[:,3].max()
# ② 取中心、算并集尺寸
cx, cy = (x0+x1)/2, (y0+y1)/2; bw, bh = (x1-x0)*w, (y1-y0)*h
# ③ 放大成正方形窗口:side = max(bw, bh) * margin(默认 1.4x,最小边长 35%)
side = max(max(bw, bh) * 1.4, 0.35 * max(w, h))
# ④ 返回归一化 xyxy → 存 bbox cache
```

训练时每个 clip 只读一次这个固定窗口, _load 里 img[iy1:iy2, ix1:ix2] 裁剪后再 resize 到 128×128;训练增强可以在窗口上做缩放/平移扰动(M11),但窗口本身是固定的。

③ 项目实例

- **+固定裁剪 = +4pt**(0.4618 → 0.5033),是除换主干外最大的单一杠杆。
- 升级到 34 层 + 帧差后,固定窗口 + 像素级预测在全量 + 双 seed 下推到 0.71144;公开 0.711 的 notebook 也把这个 trick 作为核心之一(其 fork 新版还提到 **Depth fallback**:IR 探测失败时用 Depth_Color 重新探测,我们同样保留这手)。

④ 适用边界 / 常见误区

- 误区 1:以为"检测得越准越好",于是逐帧裁剪 → 丢位移。**检测要"稳",不要"贴"**。
- 误区 2:窗口太小,把躯干/肢体裁掉 → 用 margin=1.4x 和最小边长兜底;
- 误区 3:对"人物完全不动的静态动作"(看电视),固定窗口和逐帧没差别,但这类动作本来就难——不要因此否定方法论。

⑤ 面试一句话

"我们对 YOLO 输出做 union + 1.4x 放大,生成每 clip 一个固定窗口——因为逐帧裁剪会抹平人的位移运动,而固定窗口让网络在固定坐标系里看清'人从左边走到右边'。这是我们从 0.46 涨到 0.50 的关键,也是公开 0.711 方案的核心 trick。"

第三部分:评估协议与数据治理(先修好尺子,再量身高)

M8. 跨被试评估(subject-split)与"身份捷径"

① 思想

人体动作数据里,"**谁来做的**"信息量远大于"做了什么"。如果同一个人的片段同时进训练和验证,模型只要学会"这是张三,张三在客厅走路的样子"就能拿分——这不是学会动作,是**学会认身份/认房间(身份捷径)**。跨被试任务的正确评估必须是:**同一被试的所有 clip 要么全在训练集、要么全在验证集**——逼模型去学**与身份无关**的动作表征。

② 具体实现(本项目 `src/split.py.split_by_subject`)

```
subj_to_idx: Dict[str, List[int]] = {}      # subject → clip 下标
...
rng = np.random.default_rng(seed); perm = rng.permutation(subjects)
# 18 个被试 / 3 折 = 每折 6 个被试验证
for f in range(n_folds):
    val_subjs = set(perm[start:end]); ...
    # 防泄漏断言: train 与 val 的被试集合必须不相交
    assert set(clips[i].subject for i in train_idx).isdisjoint(
        set(clips[i].subject for i in val_idx))
```

- 折叠数 = 3(18 被试 / 每折 6 人验证);
- 随机打乱按 **subject 级**(不是 clip 级)做,配合 `seed=42`;
- **防泄漏断言**写进代码,让"错误发生时立刻爆炸"成为特性。

 **小白补课(先把"训练/验证/测试 + 折"对齐):**标准流程 = 用**训练集**调参数 → 用**验证集(val)**选模型(防过拟合)→ **最后才在测试集**上看真实成绩;竞赛的 **LB(排行榜)** = 测试集的公开成绩。**交叉验证** = 把数据切几份轮着当验证集,每份叫一**折(fold)**:3 折 = 分 3 份,轮流用 1 份验证、另 2 份训练,做 3 次实验取平均——每个样本都被验证过一次,数字更可信。

本文几乎所有的 `fold0` = "第 0 折的验证结果",是条"便宜、快速"的对照线,专门用来筛实验(见 M36)。

③ 项目实例

Wave 1 用**随机划分**,本地 val 54.33% 沾沾自喜,提交 LB 却只有 32.8%——**21.5 个点的差距全是被试泄漏**。重建 subject-split 后,所有后续结论(消融/配方/关闭判断)都建立在这把"不撒谎的尺子"上。这也是为什么我们敢说"0.70+ 是真水平,不是虚高"。

④ 适用边界

- 若测试协议是"随机划分"(论文 Table 3 那类),用 subject-split 反而低估——**协议要对齐真实部署场景**;
- 关注 fold 间方差:某种划分下折间方差巨大,可能是数据太少或存在隐藏分组,要警惕(参见阶段 F 的跨折状态泄漏信号)。

⑤ 面试一句话

"随机划分会让人身份/房间泄漏进验证,本地虚高 21.5 个点;我们重建了按被试分组的 3 折交叉验证并写进防泄漏断言,从此所有结论都建立在能真实反映跨被试泛化的尺子上。"

M9. 类平衡采样(WeightedRandomSampler)——治 29.5 倍长尾

① 思想

类别长尾比 29.5(Walk=324 vs Watch_TV=11)。如果随机采样,每个 batch 里高频类占绝大多数,低频类每轮只出现

零星几次——梯度被高频类主导,少数类学不到,模型会倾向把一切判成高频类。类平衡采样 = 让每个类在训练中被"均匀地"看到——对每类样本"少的多采、多的少采",牺牲一点多数类、救活少数类。

② 具体实现(本项目 `src/dataset.py.build_balanced_sampler` ,精确原文)

```
from collections import Counter
cnt = Counter(labels)
weights = [1.0 / cnt[l] for l in labels]      # 每个样本的权重 = 1/该类样本数(反频)
num_samples = len(labels)
return torch.utils.data.WeightedRandomSampler(
    torch.tensor(weights, dtype=torch.float64), num_samples=num_samples, replacement=True)
```

- 思想映射到代码:类越少,该类的每个样本权重越大;
- `replacement=True` :有放回抽样(否则稀有类会被抽完);
- 训练循环里 `sampler` 传给 `DataLoader`;每 epoch 用 `torch.multinomial` 重新分配,样本顺序天然变化(顺带给增强带来每 epoch 随机性,见 M11);
- 推理/验证**不用**平衡采样(保持真实分布)。

③ 项目实例

主线、Thermal、骨架的训练默认都开类平衡(`--no_balanced` 才关);论文也验证了**类平衡重采样对 RGB/IMU/Skeleton 都有提升**(Fig 6a)。我们在最不平衡的骨架、40 类上尤其依赖它。

④ 适用边界

- 有放回采样会让少数类样本被**重复看到很多次**,若类特别稀少(如 11 个样本)仍有欠拟合风险——可叠加增强来扩大有效样本量;
- 若评估指标是"多数类优先"(如宏平均 $\neq$ 微平均),要按指标反推要不要平衡。

⑤ 面试一句话

"长尾比 29.5 下,我们用反频率加权的 `WeightedRandomSampler` 让每个类被均匀看到,配合有放回采样——这是让 40 类每类都能被学到的第一道防线。"

M10. 模态索引构建:discovery 模态 / 帧号配对 / 缺失兜底

① 思想

6 个模态目录结构基本一致(`<Modality>/<Action>/<Subject>/<sample>/<files>`),但**文件格式、帧号、缺失情况各不相同**。索引层要解决三个问题:① 以哪个模态为准遍历(`clip` 是谁的");② 帧号怎么对应(同相机对齐);③ 缺了怎么办(不崩、不毒)。

② 具体实现(本项目 `src/dataset.py`)

- **discovery 模态**: `build_train_index` 默认以 `Depth_Color` 遍历(它和 `IR`、`Skeleton` 同相机、结构齐);`Depth_Color` 缺失就退到 `Thermal` (0% 缺失,最稳)。一切都为了"`clip` 列表只有一个真源,不因某一模态缺

失而少算 clip"。

- **帧号配对**: 文件名里 8 位全局帧号(`_FRAME_NUM = re.compile(r"(\d{8})")`), `list_images` 按帧号建 dict; `DepthIRVideoDataset` 取 **Depth 与 IR 共有帧号**的交集 → 同相机天然对齐。
- **缺失兜底**:
 - clip 层: `__getitem__` 返回全零张量(不崩);
 - 帧层: 某帧缺失就补零;
 - `_clip_key` 用 `action_id/subject/sample` 三重 key(同一被试不同动作的 sample 名会重复, 只 key 到 `subject/sample` 不唯一)。

③ 项目实例

- 数据核查报告: Thermal 缺失 0%、Depth/IR 3.6%、IMU 4.5%、Radar 4.2%, 测试集 405 clip 全 6 模态。
- **Skeleton 的一个大坑**: 骨架在 `.../Skeleton/<Action>/<Subject>/<sample>/predictions/*.json` (**多一层 predictions/**), 首版数据报告因此误判"骨架 100% 缺失"——这就是"索引构建要先摸清真实目录结构"的教训。

④ 适用边界

索引层是"最不该出错却最容易出错"的地方: 命名规范化、缺失处理、路径层级的假设, 都要**先跑数据盘点** (`report/data_report.json`)再写死。

⑤ 面试一句话

"数据索引层我用'单真源 discovery 模态 + 帧号配对 + 三态兜底'处理 6 个异构模态, 还踩过 Skeleton 藏在 `predictions/` 子目录导致误判 100% 缺失的坑——数据工程的坑往往藏在路径假设里。"

第四部分: 训练正则与优化(把配方调对, 比换结构更重要)

M11. 数据增强作为正则(小数据的命脉)

① 思想

2891 个 clip 对 63M 参数是"极小样本无监督式"的过拟合温床。**增强 = 在训练分布附近制造合理解变体**, 等效于扩大数据量 + 注入对"不变量"的先验(翻转不变量、亮度不变量、尺度不变量.....)。对小数据任务, 增强带来的收益通常超过任何结构改动——它是**第一个被波动的旋钮**。

 **小白补课(过拟合 & 正则化)**: **过拟合** = 模型把训练数据(包括其中的随机噪声)背下来了, 换新数据就失灵——典型信号: 训练 loss 一直降、验证 acc 却停滞或下降。**正则化** = 一切"故意不让模型背太熟"的手段: 少参数、加 dropout、加数据增强、加权重衰减.....本篇的类平衡、增强、label smoothing、dropout 全是正则化家族。**数据增强是最甜的正则**: 它不是惩罚, 而是白送"同一段动作的合理解体", 等于免费扩容训练集。

② 具体实现(本项目 src/dataset.py , aug_strength 分档 + per-clip 一致)

```
# 0=无 1=温和 2=当前强(定版) 3=更强
# 2 档: flip 0.5 | bright(0.8,1.2) | contrast(0.8,1.2) | scale(0.85,1.15) | shift ±8%
do_flip = rng.random() < aug_flip
bright = rng.uniform(*aug_bright); contrast = rng.uniform(*aug_contrast)
# 亮度/对比度作用在 [0,1] 张量上(注意保护了伪彩色深度图的几何语义):
x = x * bright
x = (x - 0.5) * contrast + 0.5
x = torch.clamp(x, 0, 1)
# 窗口级:只对存在的 crop 做缩放/平移扰动(在固定窗口上小幅扰动)
if crop is not None:
    s = rng.uniform(*aug_scale); dx, dy = rng.uniform(-shift, shift) ...
    crop = 放大/平移后的归一化窗口
if do_flip: x = torch.flip(x, dims=(3,)) # 水平翻转(4/8ch 一致)
```

三个设计细节(都是会被问到的):

1. **增强参数 per-clip 一致**:一个 clip 里的 16 帧用同一组翻转/亮度/缩放参数,保证"帧间物理一致性"(不能让某些帧更亮、某些更暗——那是假象);
2. **分档(0/1/2/3)做消融**,不拍脑袋:aug0→aug1 涨 4.7pt(过拟合),aug2 全量最稳,aug3 收益饱和;
3. **保护物理语义**:伪彩色深度图里"颜色=距离",亮度对比度可动,但不能乱翻转语义(水平翻转在动作识别近似合法)。

③ 项目实例

- aug 搜索(fold0+60ep):aug0=0.6179 → aug1=0.6652 → aug2=0.6609 → aug3=0.6609;**定版 aug2**;
- 教训:aug1 在 fold 上最好(**0.6652**)但全量只用 0.652;**fold 结论不能直接映射全量**——所以定版超参必须再经全量验证;
- 增强的可迁移性:同一套增强同时用于 main/thermal/骨架(骨架另有 random_moving ,见 M18)。

④ 适用边界 / 误区

- 增强过大 → 制造训练分布里不存在的假象(把距离-颜色映射打坏);
- 增强不是为了"多样性"而多样性,要**对齐任务的不变量**(人体动作:翻转、光照、视点合理的;颜色乱变不合理);
- 每个模态的正确增强域不同:图像用翻转/亮度,骨架用旋转/平移/缩放(M18),IMU 用重力/旋转。

⑤ 面试一句话

"小数据下,增强是比任何结构更有效的正则。我们按 aug_strength 分档消融,per-clip 一致的翻转/亮度/对比度/缩放/平移——定版 aug2;同时悟到一个重要教训:fold 上最优的增强强度,在全量上未必成立,所以定版必须再过全量验证。"

M12. Label Smoothing——一次校准,喂饱集成

① 思想

标准 CE 训出的模型**过度自信**(one-hot 目标把 logits 往 $\pm\infty$ 推)。后果:① 泛化差(赌太大);② 做集成的把不同模型概率平均时,"嗓门大的"主导融合。**Label Smoothing = 把 one-hot 标签松弛**

弛: $\text{target}' = (1-\epsilon) \cdot \text{one_hot} + \epsilon/K \cdot (\text{均匀分布})$, 让模型别赌 100%, 输出更**校准**的概率。**零结构改动、一行实现、对集成友好。**

 **小白补课(先把三个词钉死: logits / one-hot / softmax)**: 分类网络最后一层对 40 个类各输出一个**原始分数 (logits)**——可以理解成"每个类的支持票数", 数值任意、可正可负。把 logits 喂进 **softmax**, 就变成 40 个加起来=1 的**概率分布**。训练的标准答案是 **one-hot**(正确类=1 其余=0), 用**交叉熵量**"模型概率分布 vs one-hot"差多远、越小越好。**label smoothing 就是把 one-hot 的 1 和 0 揉软一点(如 0.9 + 0.1·均匀)**, 逼模型别把某一类的概率赌到 100%。

② 具体实现(本项目)

```
# train_step1.py
crit = nn.CrossEntropyLoss(label_smoothing=0.1)      # ε=0.1
```

- evaluate 也用同一 label_smoothing (保持度量一致, 虽然 val 指标本质还是 argmax 准确率);
- 全量训练、thermal、骨架增强版都使用; 关掉它的对照用于验证它确实有用。

③ 项目实例

我们在配方定版里加入 label_smoothing=0.1 (与增强一起成为全量 0.71144 的配方组成之一)。它是"零风险一行代码"——因为校准过的概率, 是 --prob_avg 融合的燃料(见 M25)。

④ 适用边界

- ϵ 过大 → 模型欠自信、欠拟合(通常 0.1 是甜点);
- 若最终只做 argmax(取概率最大那类)不集成, label smoothing 收益变小但一般也无害;
- 对蒸馏场景: teacher 已软化的目标又要 smoothing 时要注意叠加效应。

⑤ 面试一句话

"我们加 0.1 的 label smoothing 来校准模型的置信度——它对最终'概率平均融合'至关重要, 因为过度自信的模型会在集成里嗓门过大、主导错误。"

M13. 调度器: StepLR vs CosineAnnealing——一个"经验要找边界"的教材

① 思想

学习率调度 = "什么时候走大步、什么时候小幅精修"。两大流派:

- **StepLR**:每隔固定步数 lr 乘以 gamma (衰减到某值后稳住)。**不会到 0。**
- **CosineAnnealing(T_max=epochs)**:余弦地从峰值退火到 0。收敛"好看",但**最后 epoch lr≈0。**
本项目踩过 **Cosine 归零陷阱**:在 50-60 epoch 的训练中,lr 被衰减到 3.94e-07(≈0),**后一半 epoch 几乎不更新**——画面是 loss 后半程纹丝不动、训练白烧。

② 具体实现(本项目)

```
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs) # ❌ 归零陷阱
sched = torch.optim.lr_scheduler.StepLR(opt, step_size=10, gamma=0.5) # ✅ 主用
```

- 后来所有训练脚本统一 StepLR(step, gamma) 或保证 lr 不退到 0;
- **但骨架线反例(阶段 E 修正)**:骨架的 transformer(DSTformer)**Cosine 反而比 StepLR 好**(0.5447 vs 0.5113)——**因为 transformer 家族(如 Swin)原文就用平滑衰减,平滑退火对它有好处。**

③ 项目实例

- 双流融合(multi-task)、MISA 等多次中招 Cosine 归零(损失后半程平、lr→3.94e-07);
- 全量训练用 Cosine 是因为**没有 val、靠退火收敛后取最后 epoch(M16)**——所以"用 Cosine 归零"在"无早停的全量"场景反而是特性(收敛到底),在"有 val 早停"场景是灾难。**同一工具,场景不同,用法不同。**

④ 适用边界

- 有 val 早停 → StepLR / 阶梯衰减(或 Cosine 但 T_max 留足余量);
- 无 val 全量靠退火选 checkpoint → Cosine 归零可作为"自动收敛"用;
- transformer/大模型微调 → 常配平滑衰减(Cosine/线性)。

⑤ 面试一句话

"我们吃过 CosineAnnealing 把 lr 衰减到 0、后半程白烧的亏,统一改 StepLR;但骨架 transformer 上 Cosine 反而更优——这教会我'经验要标注适用边界',同一旋钮在不同架构/场景下的最优位置不一样。"

M14. 预训练迁移:外观迁移 vs 结构迁移 + stem 改造

① 思想

预训练给了模型"见过几千万种运动/图像模式"的起点,小数据微调只需适配 **40 类语义**。但关键在**预训练域和我们输入的相似度**:

- **外观迁移**:输入分布接近(自然图→自然图;Thermal 3ch↔Kinetics 3ch),低层特征可直接复用,**stem 甚至可以不改**;
- **结构迁移**:输入语义错配(伪彩色深度图 ≠ 自然 RGB),但**边缘/纹理/运动检测器依然通用** → 保留 backbone、替换低层 + 微调。
本项目另一个洞察:**Depth_Color 是 jet 伪彩色深度图(不是自然 RGB)**——所以对它的预训练是**结构迁移**,不是

外观迁移;而 **Thermal(3ch)对 IG-65M 是外观迁移(100% 权重可用)**。这也是 Thermal 不需要改 stem 的原因。

② 具体实现(本项目 `src/model.py.adapt_conv3d` ,精确原文)

```
repl = nn.Conv3d(channels, conv.out_channels, conv.kernel_size, ...)
with torch.no_grad():
    repl.weight[:, :3] = conv.weight # 前 3 通道复制预训练
    repl.weight[:, 3:] = conv.weight.mean(dim=1, keepdim=True
                                          ).expand(-1, channels-3, -1, -1, -1) # 新通道=前3通道的均值!
    repl.bias.copy_(conv.bias)
```

为什么新通道用"均值"而不是随机:

- 随机初始化会让新通道的激活量级和预训练不一致,破坏网络前馈的数值稳定性;
- 用"前 3 通道权重均值"初始,新通道从一开始就有"合理量级"的输出,预训练知识不受扰动。
(另有 `adapt_conv2d` 给 TSM-ResNet18 的 `conv1` 用。)

③ 项目实例

- 视觉主线:Kinetics-400(R2+1D-18)→ IG-65M+Kinetics(R2+1D-34,自监督 6500 万 Instagram 视频 + Kinetics 微调),4ch stem via `adapt_conv3d` ;
- Thermal:R2+1D-34 3ch **完全复用 IG-65M**。
- **关键超参耦合(M15 预警)**:预训练 ≠ 免费——lr 必须小($1e-4$),否则把预训练特征冲垮(R2+1D-34 的 0.4252 教训 → $1e-4$ 重训 0.5988)。

④ 适用边界

- 预训练域与输入严重错配时(如深度图配 ImageNet 外观),收益有限,别期待白赚;
- "大预训练模型"在强约束赛道有合规风险(本项目澄清:**小型标准预训练 CNN 允许,LLM/VLM 基础模型禁止**)→ 选 IG-65M 这类中小预训练正好合规。

⑤ 面试一句话

"我们把'预训练迁移'分成外观迁移和结构迁移两类:Thermal 3ch 对 IG-65M 是外观迁移(100% 复用),伪彩色深度图是结构迁移(只保留通用低层)。stem 改造用'新通道=前 3 通道均值'初始化保持量级稳定;同时强调预训练必须配小 lr,否则会把预训练权重冲垮。"

M15. 微调超参:为什么"大模型 + 预训练"必须小学习率

① 思想

微调的本质是"在预训练找到的好起点附近做局部搜索"。lr 太大 = 一步跨越出好区域 → 预训练学到的通用特征被冲垮,退化成"从零训练"(甚至更糟,因为过了拟合期直接炸)。直觉:预训练权重是"老人的经验",小 lr 是"微调",大 lr 是"洗脑"。小网络(30M)受得了 $1e-3$,大网络(63M)+ 精细预训练(IG-65M)必须 $1e-3 \rightarrow 1e-4$ 。

② 具体实现(本项目)

- 主线全量: `--lr 1e-4 (R2+1D-34 + IG-65M 微调)`;
- 骨架(MotionBERT): `lr_backbone=1e-4 / lr_head=1e-3` (官方配方), `weight_decay=0.01`, AdamW;
- head(Dropout 0.3)与 backbone 分开训;恰当的 `weight_decay` 不侵蚀预训练权重;
- 调度器与 lr 的搭配见 M13(M15 决定"初始 lr", M13 决定"之后怎么变")。

③ 项目实例(最经典的教科书式教训)

- `R2+1D-34 + IG-65M + lr=1e-3` → **0.4252(比 18 层还差)**;
- 首判"容量过大、小数据过拟合" → **错误**;
- 复盘定位:lr 冲垮预训练 → 改 `1e-4` → **0.5988(+17.4 个点!)**。
- 一句话教训:"**容量不是问题,训练细节才是**"——定位失败要查超参,别急着下"架构不行"的结论。

④ 适用边界

- 预训练质量越高、模型越"熟",lr 应越小(IG-65M 这种精细权重尤其敏感);
- backbone 与 head 的 lr 常需解耦(head 从零练,可以大一点);
- 没有预训练、纯从头训练的小网络, `1e-3` 没问题——"**1e-4**"不是普适答案,是"预训练微调"的答案。

⑤ 面试一句话

"R2+1D-34 第一次训崩(0.4252),终极根因是 `lr=1e-3` 把 IG-65M 预训练权重'洗脑'了;降到 `1e-4` 直接 +17 个点。这教我两个原则:预训练微调用小 lr;看到'大模型反而更差'先排查超参,别轻易下'容量过大'的结论。"

M16. 早停 vs 全量训练:选 checkpoint 的哲学

① 思想

"训练到哪一步的模型最好"分场景:

- **有 val**:早停 / best-val 选点(guard 住过拟合);
- **无 val(全量)**:全部数据都拿来训练,"哪个 epoch 最好"没有客观标尺 → 两个替代方案:① 靠 **Cosine 退火收敛**,直接取最后 epoch(训练已被调度器"自动收尾");② 存中间快照备用。
全量 +45% 数据通常比任何 fold 方案更准(每折模型只见过一半被试),代价是没有 val 可调超参、只能靠 LB 试错 seed(M25)。

② 具体实现(本项目 `train_step1.py --full`)

```
if full:
    if save_path and ((ep+1) % save_every == 0 or ep == epochs-1):
        torch.save(model.state_dict(), save_path) # 定期快照 + 最后 epoch
    continue # 无 val,不 evaluate
```

配合: `--full_save_every`,多个 seed(42/2024/777)各训一个全量模型 → 推理时集成。

③ 项目实例

- fold 方案(fold0 0.6609 量级)≈ 用来决策配方;
- full 单 seed42 = 0.70646 LB;full seed42+seed777 集成 = **0.71144**;seed2024 拖累(0.686)被弃;
- **fold 结论≠全量**:aug1 fold0 最好但全量崩(0.652);所以"配方小白鼠=fold0,最终判决=全量"。

④ 适用边界

- 若任务有强噪声/伪标签参与,全量无 val 很危险(过拟合测试分布)——要留 held-out 监控(M30);
- 全量适合"配方已定稿、纯冲分数"的收尾阶段,M16 与 M25(seed 集成)通常一起用。

⑤ 面试一句话

"冲分阶段我用'全量 18 被试 +45% 数据'训练,无 val 时靠 Cosine 退火取最后 epoch、存中间快照;配合多 seed 试错集成。关键教训是 fold 小老鼠决定'配方',全量才决定'最终分数',两者不能互相替代。"

M17. 输入归一化:为什么"均值/方差用哪个域"能值 5 个点

① 思想

归一化 $(x - \text{mean}) / \text{std}$ 的作用是让输入分布落在**预训练时见到过的数值范围**。**mean/std 与预训练域不匹配 = 预训练模型睁眼看到的"世界"被平移+缩放了**,低层特征全部失真。PyTorch 预训练模型对应的归一化就是预训练域的标准:

- **ImageNet**: mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225);
- **Kinetics(R2+1D / IG-65M 域)**: mean=(0.43216, 0.394666, 0.37645), std=(0.22803, 0.22145, 0.216989)。

② 具体实现(本项目 src/dataset.py ,精确原文)

```
mean3 = (0.43216, 0.394666, 0.37645); std3 = (0.22803, 0.22145, 0.216989) # Kinetics
# IR 通道无自己的域 → 用三通道的均值;帧差通道 mean=0 std=0.25(差分近似白噪声)
self.mean = torch.tensor((*mean3, sum(mean3)/3.0, 0,0,0,0)).view(1, C, 1, 1)
self.std = torch.tensor((*std3, sum(std3)/3.0, 0.25,0.25,0.25,0.25)).view(1, C, 1, 1)
```

- ThermalVideoDataset 额外支持 --mean_std 自定义(早期误用过 ImageNet);
- 归一化放在**所有增强之后**、网络输入之前。

③ 项目实例(隐藏的大 bug)

Thermal 之前一直用 ImageNet 均值归一化,但模型是 IG-65M(Kinetics 域)→ 这是 Thermal 落后的实因之一;改成 Kinetics 后,Thermal fold0 0.5646 → **0.6053(修复 归一化 bug + 补 recipe 后总计到 0.6190)**。一个"看起来人人都会、人人都可能错"的参数,直接价值 5 个点。

④ 适用边界

- 换预训练 = 换归一化(IG-65M/Kinetics、VideoMAE/ImageNet 各一套);

- 伪彩色/灰度模态没有内置域 → 用"融合通道尺度"列(如 IR 用三通道均值),或自行统计;
- 忘了归一化 vs 归一化错域,后者更隐蔽(不报错、只降点)。

⑤ 面试一句话

"归一化域必须匹配预训练域——Thermal 误用 ImageNet 均值喂 Kinetics 模型,是它落后的实因;换成 Kinetics 归一化 + 补全 recipe 后连涨 5+ 个点。这类不报错的隐藏 bug,比报错更值得警惕。"

第五部分:骨架表示学习(特殊模态的完整方法论链)

骨架是"唯一一个不是图像/视频的模式、但被明确是强模式"——它的价值在**原生 3D 结构**(外观/身份无关的身体几何)。这部分是"表示工程"的完整教学:数据怎么定义 → 怎么预处理 → 用什么模型。

M18. 骨架数据表示:拓扑 / 坐标系 / 预处理链

① 思想

骨架是一个 clip 的 17 个 3D 关节随时间变化的序列 $[T, 17, 3]$ 。它**没有像素**,直接是"身体的几何":唯一一个"外观/身份无关"的模式。但原始坐标**充满捷径**——关节的绝对位置(站在哪)、人的体格大小(肩宽差别)、身体朝向(视角)都会污染表征。**表示工程的三个目标:去身份、去体格、去视角。**

 **小白补课(骨架 / 关节 / 拓扑是什么):**"骨架" = 用姿态估计算法(如 MMPose)从人体上挑出 **17 个关键关节**,每个点带 3D 坐标,连起来就是一根"火柴人"。**拓扑** = 这些点谁跟谁连(哪个是左肩、哪个是右肘)。**不同数据的连法不一样**:最常见两种是 **COCO-17**(2D 姿态/检测圈)和 **Human3.6M-17**(3D 动作圈)——**关节顺序和连接完全不同**(比如左右臂序号互换),所以不能用 COCO 的规则去套 H3.6M 的骨架。

② 具体实现(本项目 `src/skeleton_dataset.py`)

- **拓扑**: Human3.6M-17(0 骨盆 / 1-3 右腿 / 4-6 左腿 / 7-10 躯干头 / 11-13 左臂 / 14-16 右臂),**不是 COCO-17**(左右臂/腿的顺序、派生关节点都不同,ST-GCN 必须用这个边集);
- **坐标系**:原始 json 里 $[x, y, z] = [\text{水平}, \text{深度}, \text{垂直}]$,**数据层交换为** $[\text{水平}, \text{垂直}, \text{深度}]$ $[0, 2, 1]$;深度(z)本来就是"到地面的高度"(0~2.27m,单位米);
- **预处理链(顺序很重要)**:

```
kp = kp[:, :, [0, 2, 1]]          # ① 坐标语义规范化
kp = kp - kp[:, 0:1, :]          # ② 中心化:减骨盆 → 去绝对位置(身份/房间捷径)
shoulder = norm(kp[:, L] - kp[:, R])  # ③ 尺度归一化:除以肩宽 → 去体格差异
kp = kp / (shoulder[:, None, None] + 1e-8)
# ④ 再把关节速度(帧差)并成特征:vel[t] = kp[t+1] - kp[t]
vel = zeros_like(x); vel[1:] = x[1:] - x[:-1]
x = cat([x, vel], dim=-1)        # [T, 17, 6]
```

- 速度通道在**增强之后**算(保证坐标一致);

- 增强 random_moving :首末帧各采样一组旋转/平移/缩放、中间帧线性插值——**模拟相机平滑移动**(ST-GCN §3.6),物理合理;
- MotionBertSkeletonDataset 版本:默认 input3d=True (原生 3D [水平,垂直,深度]),并支持 norm=shoulder|torso (torso=骨盆0-颈8 的长度,比肩宽更稳:肩宽受 2 个关节噪声影响,躯干用一个长尺度)。

③ 项目实例

- **一个值几个点的表示 bug**:旧实现喂 $[x, y, \text{conf}]$ (2D + 恒 1.0 置信度),但 MotionBERT 预训练是 3D(model_pos ,H36M/AMASS)→ 第 3 通道喂常数=丢原生深度+3D 预训练浪费。改 input3d=True 后 3D 预训练才真正迁移;
- **置信度恒 1.0(退化)**:全量扫描 153 万值全为 1.0 → 去掉 conf 通道(ST-GCN 输入 7ch→6ch);MotionBERT 输入 $[x,y,\text{conf}]$ 中 conf 恒 1.0 无害,直接忽略;
- **坏帧**:骨架 3D 偶有坏帧/缺失 → _load_frames 逐 json 解析、坏文件跳过、缺失返回零张量, --clean 再做时间平滑。

④ 适用边界

- 绝对坐标信息在 cross-subject 是毒药(身份/房间捷径),但在**同一人/同房间**任务里去掉了反而丢信息——仍要按任务分;
- 归一化要"校准-free"(BHaRNet 扩展版强调:canonical 变换在噪声下更脆弱)——**归一化用全局的骨盆/肩宽/torso 是合理的,不要做依赖整段落地的逐样本复杂变换。**

⑤ 面试一句话

"骨架表示工程的本质是去身份、去体格、去视角:中心化减骨盆、肩宽/torso 归一化、random_moving 模拟相机移动;还踩了'喂 2D+常数 conf 浪费 3D 预训练'的表示 bug——表示错了,后面的模型再强也白搭。"

M19. ST-GCN:在"关节×时间"图上做卷积

① 思想

骨架天然是图:关节是节点、骨骼是边。ST-GCN(时空图卷积)把"帧序列"写成"关节×时间"的时空图,用**带可学习边权重的图卷积**在空间和时间两个维度传递信息——边的权重(哪个关节的连接对分类更重要)在训练中自动学习。这是骨架动作识别的奠基方法,后续 CTR-GCN / pose Conv3D 等都是它的衣钵。

② 具体实现(本项目,历史实现,后被 MotionBERT 取代)

- 图 = 17 节点 + **H3.6M-17 边集**(16 条边: (0,1)(1,2)(2,3)(0,4)(4,5)(5,6)(0,7)(7,8)(8,9)(9,10)(8,11)(11,12)(12,13)(8,14)(14,15)(15,16));
- 时空图卷积 = 空间邻接聚合 + 时间维普通卷积的组合;
- joint 流(坐标)+ bone 流(骨骼向量)双流, joint+bone 综合。

③ 项目实例

- ST-GCN joint+bone = **0.3479**(3 折),历史曾经是"首个超过 Thermal 37.85%"的骨架成绩(早期 39.09%);

- 但它**没有预训练**、graph topology 固定(H3.6M-17),上限被"2891 样本从头练"锁死 → 换成 MotionBERT 后骨架 0.3479 → **0.4177(+7%)**。

④ 适用边界

- ST-GCN 的边集必须匹配数据拓扑(本项目 H3.6M-17,**不能套 COCO**);
- graph 拓扑是固定的,跨数据集迁移差;现在骨架动作识别的 SOTA 是 transformer(MotionBERT 家族)或更强的 GCN 变体(CTR-GCN 通道拓扑重构)。

⑤ 面试一句话

"骨架的经典路线是 ST-GCN 这类时空图卷积——把'关节×时间'建成图、让边权重可学习;我们用正确边集实现到 0.35,但它的天花板被'没有预训练'锁死,换上 MotionBERT 后骨架直接 +7 个点。"

M20. MotionBERT / DSTformer:"骨架界的 BERT"

① 思想

ST-GCN 缺的是**大规模预训练**。MotionBERT(ICCV 2023)的核心:

1. 在**大型 3D 人体运动库(H36M + AMASS,数千万帧)**上做**掩码建模预训练**:像 BERT 遮词一样遮住部分关节/帧,让模型去重建——从而学到**通用的人体运动先验**("哪些姿态序列是合理的人")。
2. **编码器是 DSTformer**(双流 Swin 式 transformer):Swin Transformer 的分层滑窗注意力 + 关节/帧双粒度建模,在骨架领域非常强。
3. 预训练后又可继续做 2D→3D 提升 / 3D→2D 降维 / **动作识别微调(ActionNet)** 多种任务——**同一个 encoder,多任务通用**。

 **小白补课(transformer / attention 是什么):**Transformer 是 2017 年起统治 NLP、很快蔓延到视觉/骨架的架构。核心是 **Self-Attention(自注意力)**:让每个"位置"(比如每一个关节、每一帧)先"看一遍所有其他位置",算出"我该给谁多大关注度",再把它们的特征按关注度加权混合。好处:**每个位置直接和全局交换信息**——不像卷积只能看身边一小块——所以长距离关系(比如"左手和右脚怎么配合")天然能学。**DSTformer** = 在"关节"和"帧"两个维度各做一遍自注意力的骨架专用 Transformer,再叠多层。

② 具体实现(本项目 `src/motionbert/ + scripts/train_skeleton_motionbert.py`)

- 自包含复刻: `dstformer.py` (DSTformer,DimFeat=256 / MLP ratio 4 / depth 5 / heads 8 / maxlen 243,即 **MB_lite 架构**) + `action_net.py` (ActionNet 分类头);
- 权重加载 `load_pretrained_backbone` 兼容三种格式:官方预训练 `model_pos` (带/不带 module. 前缀)、ActionNet 动作权重(`backbone. / head.` 前缀)、裸 backbone;
- 按 `joints_embed.weight` 的 **out_features** **自动判断架构**(256=MB_lite,512=MB_release),比按 key 推断更可靠;
- 微调超参(官方配方):AdamW, `lr_backbone=1e-4 / lr_head=1e-3`,`weight_decay 0.01`,`head dropout 0.5`;
- 训练: `model(x)` 输入 `[N, T, 17, 3]` (原生 3D),推理 `unsqueeze(1)` 加帧维。

③ 项目实例

- MB_lite 预训练(61-82MB fp32)在前 3 折:0.4177(std 0.0108)——**预训练给骨架 +7%**;
- 增强修好后骨架到 ~0.45, torso 归一化 + clean 的 Cosine fold0 到 **0.5285~0.5447**(历史骨架最好水平);
- **跨折状态泄漏 bug(阶段 F)**:模型在 fold 循环外创建被复用 → fold1=0.9090/fold2=0.9882 的假象;修复=每折重建 model;
- **NTU 外部数据(M32)**:NTU60 预训练-微调只高 0.34pt 且体积翻倍 → 采用 MB_lite。

④ 适用边界 / 误区(值钱的点)

- **输入表示必须匹配预训练任务**:3D 预训练权重要喂原生 3D(不能喂 2D+常数);2D 版本的 MotionBERT 权重(喂 [x,y,conf])是另一套,不要混;
- transformer 家族的调度器偏好平滑衰减(见 M13 的反例);
- 骨架动作的 cross-subject 天花板受"数据里有多少不同的人"约束,别指望 0.45 直接拉到 0.7(论文随机划分 79% vs cross-subject 45%——掉幅与 RGB 一致)。

⑤ 面试一句话

"骨架线我们把 ST-GCN 换成 MotionBERT——一个在 H36M+AMASS 上做掩码建模预训练的 DSTformer,等于把'人体运动常识'搬进网络,微调后骨架 +7%;同时强调了'输入表示必须匹配预训练任务'(3D 权重配 3D 输入),并修过跨折状态泄漏 bug。"

第六部分:量化与部署(把预算花在刀刃上)

M21. 权重量化 101:per-channel 对称量化

① 思想

赛制硬约束:**推理加载的全部权重打包 <100MB**。R2+1D-34 单模 fp32=247MB,完全超限。**量化 = 用更少的 bit 表示权重,牺牲一点精度换体积**。核心思路:

- 每个权重张量太大、数值范围太宽,没法直接用低 bit;
- **找每张量的最大绝对值(max)** 做 scale,把所有权重"除到 [-1,1]",再乘到目标整数域 → $q = \text{round}(w / \text{scale})$ $\in [-2^{(b-1)}, 2^{(b-1)}-1]$ 。
- **per-channel = 每个输出通道一个 scale**:逐通道的尺度更贴合每列权重的真实分布,比"整张量一个 scale(per-tensor)"损失小得多。

 **小白补课(bit / fp32 / int8,为什么模型会"超重")**:网络的所有参数就是一大串数字。**fp32** = 每个数字用 32 位(4 字节)存一个浮点,精度高但体积大——63M 参数 × 4 字节 ≈ 250MB,超了赛制 100MB。**量化 = 用更少的位存同一个数**(8 位=1 字节/个,叫 **int8**;5 位叫 **int5**),代价是数值变粗糙(像把 3.14159 记成 3.1)。**bit 越低 → 体积越小 → 误差越大**,本篇 M23 会讲在哪一级 bit 上精度会崩。

② 具体实现(本项目 src/quantize.py.quantize_state_dict ,精确思路)

```

qmin, qmax = -(2**(bits-1)), 2**(bits-1)-1
for k, v in sd.items():
    # 只量化 float32/half 的大权重(Conv/Linear weight), ndim>=2 且 numel>=4096;
    # bias/BN/小张量保持 fp32(数量少、对精度敏感)
    if v.dtype not in (fp32, fp16) or v.ndim < 2 or v.numel() < 4096:
        keep fp32; continue
    flat = v.reshape(v.shape[0], -1) # 按输出通道展开
    amax = flat.abs().max(dim=1, keepdim=True).values.clamp_min(1e-8)
    scale = amax / qmax # 每通道一个 scale
    q = (flat / scale).round().clamp(qmin, qmax) # 对称量化

```

推理时反量化: $w \approx q * scale$ (加载量化包后反量化回 fp32 计算——我们不追求"真正的 int8 算子加速",只图"体积达标 + 精度损失小")。

③ 项目实例

- 体积账: 63M 参数的 R2+1D-34, int8(1B/参数)≈ 66MB, **int5(5bit/参数)≈ 40MB**;
- 预算重排: main int5 + thermal int5 = 80MB < 100MB  (留 20MB 余量);
- 对比: 纯 fp32 三模态 346MB 完全不可行, int8 三模态 86MB 也可行但我们最终用"两个 int5"装下**两个独立视角**。

④ 适用边界

- per-tensor vs per-channel: per-channel 精度更高, 但麻烦一点点(存每通道 scale);
- 量化不是免费的: int4 就崩(见 M23);
- 若目标是把模型跑在小硬件上, 还要考虑算子是否被加速器支持(本项目只求体积, 反量化回 fp32 推理最简单)。

⑤ 面试一句话

"100MB 约束下我做了 per-output-channel 对称量化: int8=1B/参数、int5=5bit/参数; 只量化大权重、bias/BN 保持 fp32; 反量化回 fp32 推理以保住精度。核心预算哲学是'在崩之前取最大压缩, 把省下的体积换成一个独立视角的模式'。"

M22. Bit-Packing: 把 int5/int6 真正塞进字节

① 思想

int8 用 PyTorch 原生 torch.int8 就够; 但 int5/int6 不是标准数据类型——**必须自己把 5 位/6 位整数"打包"进 uint8 字节流, 即位压缩**: 把多个低精度整数的 bit 首尾相连塞进连续字节, _packbits / _unpackbits 是标准实现。

② 具体实现(本项目 src/quantize.py._pack_bits/_unpack_bits, 精确思路)

```
def _pack_bits(values, bits):          # values: uint 数组(0..2^bits-1)
    bit_matrix = ((values[:,None] >> arange(bits)) & 1)  # 每位拆开 [N, bits]
    flat = bit_matrix.reshape(-1); flat = pad 到 8 的倍数
    return np.packbits(flat)           # 压成字节流

def _unpack_bits(packed, bits, n_values):
    flat = np.unpackbits(packed)[:n_values*bits]
    bit_matrix = flat.reshape(n_values, bits)
    return (bit_matrix * (1 << arange(bits))).sum(axis=1) # 还原整数
```

量化侧:int<8 时 $q_{\text{unsigned}} = (q - q_{\text{min}})$ 变非负整数 → 打包;反量化侧 $\text{unpack} \rightarrow + q_{\text{min}} \rightarrow \times \text{scale}$ 。打包的字节存成 torch.uint8 一起 torch.save。体积公式:1 个参数占 $\text{bits}/8$ 字节。

③ 项目实例

- int5 → R2+1D-34 ≈ 40MB; int4 更是 32MB(但崩了,见 M23);
- submission_fusion_mT.csv (保底提交 0.73)的 80MB = main int5 + thermal int5,正是 bit-packing 的产物;
- 打包脚本 scripts/quantize_pack.py 会自动统计总大小并检查 <100MB,超了会提示三条退路(合并多折、减模态、降 bits)。

④ 适用边界

- bit<8 时反量化有额外的 unpack 开销(推理慢),本项目接受(容量是关键约束);
- 位压缩的数值表示必须"无符号平移"(q - qmin 再 pack),否则负数 bit 表示会乱;
- 节省边界:0.49 字节/参数的收益在 int4 处就到了"精度崩"悬崖(M23)。

⑤ 面试一句话

"实现上我把 int5/int6 用位打包(_packbits/_unpackbits)压进 uint8 字节流,一个参数只占 bits/8 字节;打包的同时用 scale 元信息保证反量化无损还原,最终两个 int5 模型恰好 80MB,合规还有余量。"

M23. 精度-体积权衡:int4 为什么会崩 + 预算重排

① 思想

量化的收益是体积线性下降,但代价是非线性(存在"悬崖"):当 bit 数低到"量化步长 > 分类边界敏感度"时,不同类之间原本就接近的 logits 差异被一步碾平 → 分类错误率暴涨。对 R2Plus1D 这种深度网络,int4 就是那个悬崖。

② 具体实现(本项目,用提交做实验)

- 提交实验矩阵:

方案	体积	LB
main int5 + thermal int5(80MB)	80MB	0.73
main int4 双 seed + thermal int4(96MB)	96MB	0.71 比 0.73 还低

- 判定:int4 对 R2Plus1D 精度崩(量化误差放大到分类崩溃)→ 铁律:R2Plus1D 提交级最低 int5。
- 预算重排结论:"1×main(int5) + 1×thermal(int5) 独立视角" > "2×main 双 seed int4"——在体积允许内,优先放信息独立、质量高的模型,而不是把预算摊薄去放更多同质模型。

③ 项目实例

int4 崩($0.71 < 0.73$)是"量化精度非线性"的直接证据;它同时帮我们钉死了 **int5 为提交级底线**。这也是我们最终 0.73 保底配置的由来: submission_fusion_mT.csv (main_s42 int5 + thermal_s42 int5 + prob 平均 + flip TTA)。

④ 适用边界

- 小模型/浅网络(如骨架 ActionNet)可能 int8/int5 都没问题,但**深度 CNN(63M)对低 bit 更敏感**;
- 有"量化感知训练(QAT)"可以缓冲悬崖,但我们用普通的"训练后量化(PTQ)+ int5"够了——因为目标是提交体积,不是跑在真实 int 硬件上。

⑤ 面试一句话

"我们实测 int4 会把 R2Plus1D 的精度打崩($0.71 < 0.73$),因为量化步长碾平了类间 logits 差异——所以定铁律'提交级最低 int5'。预算哲学是'在崩之前取最大压缩,用省下的体积放一个独立视角的模式,而不是放更多同质模型'。"

第七部分:多模态融合与集成(把鸡蛋放进正确的篮子)

M24. 多模态融合层次全景:早 / 特征 / 晚 / 决策

① 思想

多模态融合不是一个"放一起就行"的动作,而是一条从输入到决策的连续谱,越靠前耦合越紧、需要的对齐越硬,越靠后越稳、越容易增量扩展:

- **数据级(早融合)**:输入端拼通道——前提是模态**硬对齐**(同相机同帧同坐标系);
- **特征级**:各模态各自编码后,在网络中间做 concat/attention/门控——需要配对数据、特征分布接近;
- **决策级(晚融合/集成)**:各模态独立出 logits/prob,再加权/平均——**最稳、最抗错对齐、最容易分头优化**。

② 具体实现(本项目)

- **早融合(在用)**: DepthIRVideoDataset 把 Depth_Color(3ch)+IR(1ch)按帧号拼成 **4ch**(frame_diff 时再并 4ch 帧差 → 8ch)→ 一个 R2+1D-34 同时看"深度几何 + 红外剪影";
- **特征级(试过,关闭)**: dual_fusion (concat + MLP,fold0 $0.6114 <$ 决策级 0.73)、MISA(0.3692)——**小数据上特征级过拟合**;
- **决策级(最终在用)**:main / thermal 各自独立推理 → prob 平均 + flip TTA → 0.73 ; softmax(val_acc) 的 --auto_weight 思路也是决策级(但最终等权概率平均最稳)。

③ 项目实例(选择规则的完整性)

层次	前提	本项目	结论
早融合	硬对齐	Depth+IR 4ch	✅ 主力
特征级	配对+够数据	dual_fusion / MISA	❌ 0.61 / 0.37
决策级	单模态达标	main+thermal prob 平均	✅ 0.73

④ 适用边界

- 没对齐就早融合 = 制造垃圾输入(热像 2D 骨架跨相机错位,早期因此失败);
- 特征级要"数据量足够撑起交互参数",2891 样本撑不起;
- 决策级是"保底万能",但要求各模态先独立达标 + 验证互补(见 M26)。

⑤ 面试一句话

"多模态融合要按'硬对齐程度'选层次:深度+红外同相机所以早融合成 4ch;特征级融合在 2891 样本上过拟合被关闭;最终用决策级概率平均——因为最稳、最能增量扩展,也让我们能'单模态先达标、再验证互补、最后才融'。"

M25. 集成:logits 平均 / prob 平均 / SWA / seed——四选一的学问

① 思想

集成 = 用"多个模型的输出"换鲁棒性。四种做法,风险完全不同:

- logits 平均:直接加 logits——不同模型 logits 的尺度/偏置不同(嗓门大的主导),不稳;
- prob 平均:先 softmax 再平均——统一成概率尺度,稳,推荐;
- SWA(权重平均):训练后把多个模型的权重做平均——把不同的'山'平均成'平原',决策边界模糊,高风险;
- seed/fold 集成:不同 seed/不同折训练的模型,输出概率平均——只有当错误互补时才有增益(M26)。

② 具体实现(本项目 scripts/ensemble_inference.py)

```
if args.prob_avg:
    fused = zeros((N, 40))
    for w, l in collected:
        p = 1 - l.max(axis=1, keepdim=True)      # log-softmax 数值稳定
        p = exp(p); p = p / p.sum(axis=1, keepdim=True)
        fused += w * p                            # 加权概率平均
else: # logits 平均
    fused += w * l
```

多折/多 seed:每个 checkpoint 都 load_state 出来独立推理、把 logits 累加、再除以 checkpoint 数 → $\text{logits} / n_{\text{forward}}$ 。

③ 项目实例(最惊险的教训)

- **SWA 塌缩(阶段 G)**:三折权重 --average 合并提交 → 测试预测塌缩到 26 类(正常 38-39)→ **LB 0.39**;改为"每折独立打包 + 推理时 logits 平均" → **0.62**;
- **prob 平均 vs logit 平均**:融合 two 模型实测用 --prob_avg (softmax 后加)更稳,不同模型 logits 尺度差异大;
- **seed 集成**:seed42 0.706 + seed777 集成 0.71144;seed2024 拖累(0.686)→ **只集成不拖累的 seed**。

④ 适用边界 / 铁律

- **绝不用权重平均(SWA)做多折集成**(决策边界模糊→塌缩);
- 集成前验证"错误互补"(见 M26)——同视角多个模型更多是压方差、跨视角才是补盲区;
- 集成有体积成本(100MB 约束):多一个模型就得量化更狠或砍别的(M23)。

⑤ 面试一句话

"集成我分清了四种做法:logits 平均会被'嗓门大'的模型主导,prob 平均最稳;SWA 权重平均实测造成预测塌缩(LB 0.39→修复后 0.62),所以铁律是多折只用'每折独立推理+概率平均';seed 集成只在错误互补时才有增益,要'只集成不拖累'的 seed。"

M26. Oracle / 互补性分析:先算清"融合天花板"再投入

① 思想

融合值不值得做,不取决于"每个模态多准",而取决于**两模态的错误分布是否互补**。**Oracle(神谕)分析**:假设每个样本都能"完美选对最好的模态",融合能拿到多高?如果 oracle 只比最强单模态高一点点 → **融合空间很小,别硬融**。配套指标:rescue(副模态救回主模态多少错)、corruption(引入多少新错)。

② 具体实现(本项目 scripts/fusion_validate.py)

- 在**同一 val 折**上用同折 ckpt 推理各模态(logits 平均);
- 输出:各模态单独 acc、各种融合方式的 acc、**oracle 上限**、"**融合救回 main 的错**"数量;还支持置信门控(低置信才查 aux)等变体;
- 判定口径:只看"rescue > 0 且 corruption 小"的模态才值得融。

③ 项目实例(LB 级铁证)

- main(0.71)+ skeleton(0.45)与 main+thermal:rescue 0.18~0.25 / oracle 0.68~0.71 —— **oracle 本身 ≈ main 的 0.71** → 朴素加权只会拖累(无泄漏 val 0.566 < main 0.59);
- 正是这套分析让我们从"**强行三模态加权**"转向"**朴素 prob 平均 + 只融真正独立的 Thermal**" → 0.73。

④ 适用边界

- oracle 是"完美选择"的上界,现实只能接近,所以"oracle 提升很小"= 明确的"别做融合";

- 用"没见过的 val 折叠"的模型做拟合(见过 val 的模型会给出 1.0 假象,见 M27 与阶段 J)。

⑤ 面试一句话

"融合前我先做 oracle/互补性分析:在 val 上算'每样本选最佳模态的上限'、rescue 和 corruption——结果 oracle≈main 本身,说明融合空间极小,强行加权只会拖累。这让我把投入转向'造真正独立的 Thermal 视角',最终才涨到 0.73。"

M27. 门控 / 类感知 / 动态加权融合(试过并关闭:为什么失败)

① 思想

"融合"的更高级形式:**动态地**决定每个样本/每个类该信谁——类感知(每类一个模态权重)、置信门控(main 低置信才查副模态)、样本级动态加权。听起来很美,但在小 val 上有个死穴:**你用什么数据去"学"这些权重?**一旦用"被模型见过的 val"拟合,权重就是"记忆参数"(val acc=1.0 的假象),到测试直接失效。

② 具体实现(本项目 scripts/fusion_class_gated.py / dynamic_fusion.py)

- 类感知权重:在 val 上统计"每类哪个模态最准" → 每类一个权重向量 → 分类时按类选模态;
- 置信门控: main 的 $\max\text{-prob} < \tau$ 的样本才去查 thermal(只救不伤);
- 样本级动态加权:按 max-softmax 置信度实时调两个模态的权重。

③ 项目实例(连续三次低于朴素平均)

- 类感知/置信门控:val_acc **1.0000(假象!)**,测试 LB **0.64 < 朴素平均 0.73**;
- 动态加权:0.65671 < 0.73;
- 结论:main+thermal 是同质视觉模态,错误重叠高,**任何"学参数"的融合都是在小 val 上过拟合**;朴素概率平均不可超越就是这么来的。

④ 适用边界

- 只有当"模态高度互补"时(异质模态、且每类都有清晰的强模态)动态融合才有空间——**先过 M26 的 oracle 门槛再谈参数量化融合**;
- 拟合权重的数据必须"模型从未见过"(fold-out),否则就是记忆参数。

⑤ 面试一句话

"我们把类感知、置信门控、动态加权三种'有参数'的融合都在 val 上拟合过——全部失败(val 假象 1.0、LB 0.64<0.73)。根因是'见过 val 的模型拟合出的权重是记忆参数'。这是'花哨方法<朴素平均'三连败里最典型的一个,它强化了我们'先验证互补、再谈学参数'的纪律。"

M28. TTA(测试时增强):Flip 免费涨点,Time 版本踩过坑

① 思想

TTA = 推理时对输入做多种合法变换、取平均——**用推理算力换稳健性,不增加模型体积**。动作识别里最合法的是**水平翻转**(很多动作镜像后仍是同/近似同类)、**时间采样**的多种窗口(对采样抖动稳健)。

② 具体实现(本项目 `src/ensemble_inference.py` + `dataset.sample_offset`)

- **Flip TTA**: `out = model(x) + model(torch.flip(x, dims=(-1,)))` —— 原图 + 水平翻转各一次,logits 相加;
- **Time TTA**: `--time_tta N` → 对 clip 用 N 个不同的起点偏移(0~1)均匀采样窗口,各自推理再平均;实现上 `sample_offset ∈ [0,1]` 控制窗口起点。

③ 项目实例

- **Flip TTA**:0.73 保底配置的一部分(与 prob 平均一起),**稳赚不赔**;
- **Time TTA(翻车教训)**: `--time_tta 3` 用连续 16 帧窗口(10fps 只覆盖 1.6s 局部快照)——**丢全 clip 的时间多样性**,0.73 → **0.65174(-8pt)** 已回退;
- 教训:**视频 TTA 窗口必须"段内均匀/带偏移覆盖全程"**,不能用连续帧窗口——视频模型(R2+1D)不能丢掉全局时间视角。

④ 适用边界

- Flip 对非镜像对称的动作有理论风险(但实践证明利大于弊);
- Time TTA 只对"采样抖动敏感 + 时间结构被保留"的模型有效;clip 太短时窗口没有多样性,TTA 毫无意义;
- TTA 都要在 val 上先对拍,再上提交。

⑤ 面试一句话

"我们用 Flip TTA 免费稳涨点;但时间 TTA 踩过坑——连续 16 帧窗口只覆盖 1.6s 局部快照,把 0.73 拖到 0.65,说明视频模型不能丢全局时间视角,TTA 窗口必须段内均匀覆盖全程。"

第八部分:半监督与表征学习(试过并关闭,但要能讲透)

这一部分的每个方法都"死"在了本项目的小数据 + 同质模态条件下。会讲"为什么失败"比会讲"怎么实现"更值钱。每个方法仍按"思想/实现/实例/边界/面试"展开。

M29. 对比学习与 CLIP 范式(InfoNCE)

① 思想

CLIP 干的事:让两种视角/模态对"同一物体/同一语义"的特征一致(batch 内互为正样本),而对"不同物体"互斥(负样

本),用 InfoNCE 交叉熵拉近正对、推开负对。目标:学到**模态不变的语义表征**。论文 §6.1.3 明确说对比学习能提升 cross-subject 泛化——因为"动作语义"是跨身份不变的,对比学习等于把身份/房间噪声当负样本推开。

 **小白补课(对比学习 / 正样本对 / 负样本对)**:想象一个"找朋友"游戏:**正样本对** = 同一段动作的两种"视角"(比如 Thermal 和 Depth+IR 拍的是同一段喝水)→ 要让它们的特征**互相靠近**;**负样本对** = 不同 clip 的样本 → 要**互相推开**。**InfoNCE** 就是这道题的损失:正对相似度越大、负小越好。这样学出来的特征会变成"不随传感器视角变"的动作语义,理论上更抗跨被试。**CLIP** 就是把图"和文"拉齐的著名实现。

② 具体实现(本项目 scripts/train_contrastive.py)

```
def info_nce(z1, z2, temperature=0.07):
    z1, z2 = F.normalize(z1, dim=-1), F.normalize(z2, dim=-1)
    sim = z1 @ z2.T / temperature           # [B, B] 相似度矩阵
    labels = torch.arange(z1.shape[0])      # 对角线=正对
    return (F.cross_entropy(sim, labels) + F.cross_entropy(sim.T, labels)) / 2 # 对称 InfoNCE
# 总损失: CE(main) + CE(aux) + λ·InfoNCE(z_main, z_aux)
```

- **PairDataset**: main[i] 与 thermal[i] (或 skeleton)同 index 配对——同一个 clip 的两个视角天然正对;
- main-encoder 视频(R2+1D-34)、aux-encoder 可以是 thermal(双视频)或 skeleton(SkeletonEncoder,1D-CNN)、或对比 CE + λ·InfoNCE。

③ 项目实例(两个关闭)

- **main↔thermal**:s1 配方 fold0=0.6771(+1.2%)但 **s2 配方 fold0=0.6437(-1.7%)** → 3 折 mean 0.6475 < 基线 → 关闭;
- **main↔skeleton**:nce 对齐在学(2.0→0.25)但 **main val 零转移(0.6609=基线)** → 关闭;
- 根因(通用):**弱 aux 监督强 main**,辅助损失的梯度与主分类损失在小数据上互相撕扯——辅助信号的质量配不上主任务时,对比学习是负优化。

④ 适用边界

- 对比学习要求"辅助视角信息量足够"(0.56 的 thermal 勉强、0.4513 的 skeleton 不够);
- 负样本选择决定质量:batch 内负样本 vs 困难负样本,前者简单后者强;
- 论文的 +2~5% 收益是在"增大 batch/更多负样本 + 预训练对比"前提下,2891 样本的 batch 太小。

⑤ 面试一句话

"我们用对称 InfoNCE 让 main 和 thermal/skeleton 做跨模态对齐,想按论文路线提升 cross-subject;但实测弱辅助监督强主模型反而负收益(-1.7%)或零转移——结论:对比学习的收益建立在'辅助视角信息量足够 + 足够多的负样本'上,这两条我们的小数据都不满足,所以关闭。"

M30. 伪标签与自训练 / Co-training

① 思想

测试集 405 个 clip 是**未标注的、但与训练集同分布**的数据——官方允许作为自训练素材。伪标签 = 先用现有模型给测试打“软/硬标签”,再拿它(部分可信的)回炉训练。**风险是噪声**:伪标签错的样本会带偏模型 → 必须**严格筛选 + 低权重 + 防漂移**。

② 具体实现(本项目 `scripts/pseudo_label_selftrain.py`)

- **Stage 1(pseudo)**:用全量 main+thermal(seed42)对 405 测试打伪标签 → **Co-training 一致性筛选**(两个模型 top1 一致才采信)+ **动态 top-K 置信度**(每类按置信度排、取最可信 K 个)→ `pseudo_labels.json`;
- **Stage 2(selftrain)**:fold0 训练集(真标签 + 伪标签测试子集)**混合重训** → fold0 val 对比;伪样本用**低训练权重**(可靠性加权)、held-out val 监控、单轮自训练防漂移。

③ 项目实例

- fold0 验证:thermal 0.6275 / main 0.6566(相对基线微降);
- **全量版(唯一没烧 LB 的合法大杠杆)**: `pseudo_label_selftrain_full.sbatch` —— 融合约 20 万 + 烧一次 spare 提交才敢决策(伪标签价值针对“测试分布”,fold0 是训练分布测不出来);
- 成本收益:fold0 微降说明**同分布样本增广对训练分布的 val 没帮助**,只有对“测试分布”的 LB 才可能有用——这就是为什么必须烧 LB 试。

④ 适用边界 / 误区

- 伪标签是“用噪声换更多数据”:噪声大 = 漂移,必须“宁缺毋滥”;
- 筛选阈值/权重/top-K 都要 fold0 早停防过拟合;
- **fold0 验证不出伪标签价值**(它针对测试分布),只能烧 LB 试错——这是方法论本身的盲区。

⑤ 面试一句话

“伪标签自训练我严格做 Co-training 一致性筛选 + 动态 top-K + 低权重 + 单轮防漂移;fold0 上它微降,因为伪标签价值针对性‘测试分布’而 fold0 是训练分布——所以这是个‘只能烧 LB 验证’的合法大杠杆,我们把它留作最后冲刺。”

M31. 知识蒸馏(teacher→student)

① 思想

蒸馏 = 用强模型的**软标签**(含类间相似度信息)训练弱模型,让弱模型“学会强模型的判断逻辑”,通常比硬标签更能传递细粒度语义。动机:救骨架(0.45 → 希望 0.5+),或让 main 从 thermal 获得独立视角的软监督。

② 具体实现(本项目 `scripts/train_distill_thermal.py` 等)

- teacher:main(R2+1D-34,val 0.71)软标签;student:thermal(或骨架)用自己的数据和 teacher 的 soft label 训练(加权 $\lambda \cdot \text{KL}(\text{student}, \text{teacher_soft}) + (1-\lambda) \cdot \text{CE}$)。

③ 项目实例(判断,未成线)

- main→skeleton:即使把 0.45 提到 0.5+,骨架错误仍与 main **高度重叠**(同视觉/几何信息)→ **依然拖累**;
- 结论(11.3):蒸馏有价值的前提 = **弱模态提升后必须与 main 错误互补**;不满足就是"救起来也白救" → 关闭。

④ 适用边界

- 蒸馏最适用"student 与 teacher 视角互补"(不同输入源/不同信息组)的情况;
- 同源强压弱(0.71→0.45)大概率只是把 teacher 的错误分布也教给学生。

⑤ 面试一句话

"我评估过用 main 软标签蒸馏骨架:即使骨架从 0.45 提到 0.5,只要它和 main 错误高度重叠,救起来也白救——所以蒸馏的前提是'student 与 teacher 视角互补',我们的骨架不满足,关闭。"

M32. 外部数据迁移(NTU-60 骨架)

① 思想

2891 个骨架样本太少,而 **NTU-60 有 37,920 个骨架样本(40 类日常动作,与赛题高度重叠)**——"借"同域外部数据来增强是合法且常见的手段。但跨数据集有代价:**关节拓扑要映射(25→17)、类要对齐、体积会涨**。

② 具体实现(本项目 `src/ntu_dataset.py` + 训练脚本)

- **拓扑映射**:NTU 25 关节 → H3.6M-17(NTU60 A001-A040 单人 40 类,头 40 类语义与我们高度重叠);
- 两种用法:
 - **multi-task**:共享 MotionBERT backbone + 我们 40 类头 + NTU 40 类头
(`train_skeleton_multitask.py`, fold0 0.5264);
 - **混合增强**:NTU 清洗分类(A001-A040 中 13 类语义一致的)直接混入训练集
(`train_skeleton_augmented.py`);
- 文件与坑:NTU 是 `.skeleton` 文件序列, `find` 遍历(ls 展开 5.6 万参数会 Argument list too long);A041-A060 是双人交互必须过滤(否则交叉熵越界)。

③ 项目实例

- NTU 预训练-微调:0.4211 只比 MB_lite 高 **0.34pt**,且权重体积翻倍(188MB)超约束 → **不采用**;
- 混合/多任务:提升有限且不稳定 → 关闭;
- 结论:**外部同域数据收益被'跨数据集噪声 + 体积成本'吃掉了**,不如把预算花在"把主数据 train 好 + 独立视角"上。

④ 适用边界

- 外部数据要**先验证同域性**(类别相似度、动作分布)——NTU 日常动作与赛题高度重叠才值得试;
- 体积是硬约束:多一个 NTU 权重要想清楚占多少 100MB 预算;
- 跨数据集归一化(关节定义、帧率、相机)必须统一,否则引入噪声。

⑤ 面试一句话

"我尝试用 NTU-60 外部骨架增强我们的小数据,做了 25→17 拓扑映射、multi-task 和混合训练——但收益只有 +0.34pt 还被体积翻倍吃掉,关闭。这告诉我:外部数据要先验证同域性,否则'借来的噪声'会吃掉收益。"

第九部分:损失工程与旁路探索(在损失层面找 0.7→0.8 的跳板)

M33. 损失函数家族:CE / Label Smoothing / ArcFace / CenterLoss

① 思想

我们大部分实验用 **CE**(交叉熵),因为它简单、可校准、配 label smoothing 更稳。但要冲 cross-subject 高分,理论上有更强的损失:cross-subject 本质 = **跨身份泛化**(人脸/ReID 的核心问题),而**度量学习损失** (**ArcFace/CosFace/Center Loss**)正是为"**类内紧凑 + 类间分离**"量身定制——把人脸识别那套"同一个身份内部紧凑"泛化成"同一个动作内部紧凑、不同动作分离"。

 **小白补课(度量学习 / 类内紧凑 / 类间分离 / ArcFace)**:普通分类只要求"把类判对",不管同类样本的特征挤不挤。**度量学习** 额外要求两件事:同一个动作的所有样本在特征空间里**尽量聚成一团(类内紧凑)**,不同动作的团**互相远离(类间分离)**——这正是"跨身份"任务(人脸识别最典型)的核心诉求。**ArcFace** = 在分类头里加一个"角度间隔",逼模型必须把同类拉得足够紧才能分辨,是真实战王牌。

② 具体实现(本项目 `scripts/train_main_arcface.py` / `train_main_centerloss.py`)

- **Center Loss**:在 CE 之外加 $\lambda \cdot \sum ||f_i - c_y||^2$ (拉每个特征靠近其类中心);
- **ArcFace**:把 softmax 的 logit 变成一个"带角度间隔"的版本 $s \cdot \cos(\theta_y + m)$ ——类内角度更紧、类间角度更分,是人脸 SOTA 标配;
- 本项目把它们接到 R2+1D-34 的 head 前面(feature → 投影到单位球 → ArcFace 损失 + CE)。

③ 项目实例(状态)

- CenterLoss:中性(关闭);
- **ArcFace(进行中)**:判据 fold0 > 0.6609(CE 基线)→ 才转全量冲 0.8;这是当前 0.7→0.8 跳板里最有理论依据的空白(很多队伍靠它在 0.8+);
- 主要疑虑:2891 样本对 metric learning 偏少,类中心估计会噪声大 → 所以判据说得很保守。

④ 适用边界

- 度量学习需要每类有足够样本估计中心(长尾 29.5 的稀有类危险);

- ArcFace 的 m/s 超参对分类难度很敏感;
- 已在"类别平衡 + 增强"基础上再加才公平(否则分不清是损失还是别的功劳)。

⑤ 面试一句话

"大部分实验我们在 CE 上用 label smoothing 校准。冲 0.8 的跳板我押 ArcFace——因为 cross-subject 本质是跨身份泛化,度量学习(类内紧凑+类间分离)是 ReID 的核心武器;目前 fold0 判据待验证。"

M34. 辅助位姿回归头(auxpose / MPJPE)——为什么是"负向正则"

① 思想

一个听起来很妙的想法:让视频主网络额外回归 3D 骨架(MPJPE 损失),把"人体几何"作为监督信号注入,测试时摘掉辅助头(零开销)。这样 main 岂不学到"身体结构"?论文上这类 multitask/auxiliary pose 有先例(PoseConv3D 等)。

② 具体实现(本项目 `src/pose_aux.py` + `scripts/train_auxpose.py`)

- PoseAlignedVideoDataset :视频帧 $\leftrightarrow$ 对齐骨架帧(跨相机按帧比例对齐,thermal 25fps vs skeleton 10fps $\rightarrow$ $\text{round}(j*(n_s-1)/(n_t-1))$,缺失帧 mask=0);
- 结构:encoder 的 layer3/layer4 特征 $\rightarrow$ 逐帧回归 $[T', 17, 3]$;损失 = $CE + \lambda \cdot \text{MPJPE}(\text{预测 } 3D, \text{真值 } 3D)$;测试摘头。

③ 项目实例(两连败)

- v1 注入 **main**(Depth+IR):-0.54pt;
- v2 注入 **thermal**(`auxpose_thermal.sbatch`): $\lambda=0.05 \rightarrow$ fold0 0.5966(-3.7pt); $\lambda=0.02 \rightarrow$ 0.6211(-1.3pt); λ 越大越差;
- 根因:MPJPE 收敛到 ~5.8 而非 0(比例对齐仍有噪声)+ 分散了分类注意力 $\rightarrow$ 骨架几何监督是负向正则。

④ 适用边界

- 辅助回归头的收益前提:辅助任务是"分类的中间产物"且监督质量高(骨架有噪声时就不是);
- 想在视频网络里用骨架,更稳的路线是"对比/一致性正则"而非硬回归。

⑤ 面试一句话

"我们试过给视频主网络加 3D 回归头($CE + \lambda \cdot \text{MPJPE}$)让骨架几何监督 main 和 thermal——两连败, λ 越大越差(最多 -3.7pt)。根因是骨架本身有噪声、MPJPE 收敛不满 + 分散分类注意力:辅助监督质量配不上主任务时就是负向正则,该关闭。"

M35. v3 bounded-prior:推理侧"类偏置修正"(在用,零训练)

① 思想

40 类长尾 29.5 倍 → 模型天然对高频类有偏置:测试集上预测的 soft-count(各类预测概率之和)会显著不均 (imbalance ≥ 2 就是危险信号)。给 logits 加一个"朝均匀先验"的调整,惩罚模型过度偏爱高频类——类似 logit 校正(class prior 校正)。bounded = 有限度地修正:只翻低置信度的预测,且设硬上限,防止"修正过头"。

② 具体实现(本项目 scripts/ensemble_inference.py.safe_prior_decode)

```
soft_counts = probabilities.sum(axis=0)
adjustment =  $\lambda$  * (log(target) - log(soft_counts.clamp_min(1e-9))) #  $\lambda=0.30$ 
adjusted = log(p) + adjustment[None, :]
candidate = adjusted.argmax(axis=1) # 修正后的预测
flip = (candidate != base) & (conf < 0.55) # 只翻低置信度(<0.55)
# 设最大翻转比例 cap 6%,超过时按"修正增益"排序只翻最值得的
budget = int(0.06 * rows)
keep = flip_rows[argsort(-gain)[:budget]]
```

- 触发条件 = soft-count imbalance ≥ 2 (不满足说明模型没偏,别动);
- 输出 *_prior.csv 作side file(主提交仍是 argmax),只在有 spare 提交时烧一次 LB 验证。

③ 项目实例

- 2026-08-26 更新:fork 的公开 notebook 更新版确认这种修正存在,且 v2 damped-Sinkhorn 平衡曾丢 LB(修正过头);
- 我们实测典型 +2.3% / 最差 -1.7% —— 低风险高概率有益,是"推理侧零训练"的最后一手。

④ 适用边界

- class prior 修正是把双刃剑:测试分布 $\neq$ 训练分布时,把测试往"均匀"拉可能对也可能错——所以带 conf-gate + cap;
- 不能随意调 λ /frame/max_flip;每个超参都要在 val 上对拍;
- 它与"训练侧类平衡采样(M9)"是两回事:一个治"训练时稀有类被埋没",一个治"推理时高频类被偏爱"。

⑤ 面试一句话

"最后一个推理侧零训练类偏置修正 v3 bounded-prior:量 soft-count imbalance,朝均匀先验调 logit、只翻低置信(low<0.55)且 cap 6%,典型 +2.3%。它是'先测量再修正、修正要有限度'的代表——和训练侧类平衡采样是配套的前后端。"

第十部分:实验决策框架(比任何技巧都重要的方法论)

M36. fold0 判据 / 不烧 LB / 保底不动——怎么管住"实验的嘴"

① 思想

竞赛里**每次提交都是一枚筹码**。科学实验应该把"探索"和"定稿"分开:

- **探索(小白鼠)**:一切新方法先用 **fold0 快速对照**(几小时跑完),超过基线(如 main 0.6609)才可能升级;
- **定稿(判决)**:只有真的到达"提交级"(全量 + 多 seed + 量化)的模型才进正式提交;
- **保底绝不动**:已经打下的 0.73 配置(种子提交文件)视为"铁底",实验失败随时能退回去。

② 具体实现(本项目)

- EXECUTION_STEPS.md 明确:0.73 保底绝不动;所有实验 fold0/val 判据,不烧主 LB;
- 每个 sbatch 入口 + 一行判据(如"fold0 >0.66 留 / <0.66 关");
- TODO_experiments.md 给每个待办标"优先级 + 判据 + 状态 + 失败即关闭";
- 面向失败文化:关闭不是失败,是"用最低成本拿到'这条线不值得'的证据"。

③ 项目实例

- 双流融合 fold0 0.6114 < 0.66 → 关闭,不再烧正式 LB;
- ArcFace 判据未过 → 不转全量;
- 伪标签全量是"唯一没烧 LB 的合法大杠杆",宁可最后烧一次 spare。

④ 适用边界

- fold0 判据的局限:fold0 不一定代表全量(见 M16 的 aug1 教训)——所以最终判断必须全量再验;
- 代价/收益权衡:实验便宜(几小时 GPU)但 LB 筹码稀缺——这套纪律的数学根基就在这。

⑤ 面试一句话

"我管理实验靠三条纪律:新方法一律先 fold0 对照超过基线才升级;保底提交绝不动;每个实验都写成'判据+状态+失败即关闭'。这让我能用最低成本拿走过一打方法的'不值得'证据而不烧掉正式提交——关闭也是一种能力,因为它是系统排除法的产物。"

M37. 实验管理与可复现工程(SLURM / 记忆 / 结果归因)

① 思想

一个能打到 0.73 的项目,胜负手很大程度在工程:实验可复现(一份 sbatch = 一个实验)、结果可归因(每个数字变化

能追溯到一个旋钮)、上下文可延续(数据事实/教训写进记忆,换人换轮不丢)。

② 具体实现(本项目)

- **实验即代码**:每个实验一个 `scripts/*.sbatch` (SLURM 提交),锁 `--gres=gpu:A100:1` (RTX5090 在 `batch32+8ch` 会 OOM);
- **一份代码两处跑**:本地只放样例、服务器跑全量——用"数据路径 + flag"参数化;
- **HF 镜像** `HF_ENDPOINT=https://hf-mirror.com` + 大文件下载重试(DNS 抖动);
- **Windows→Linux 换行坑**:Windows 写的 `.sh/.sbatch` 是 CRLF,服务器 bash 报 `unexpected end of file` → 上传前统一转 LF;
- **防御性断言**:防泄漏 `isdisjoint`、提交行数=405、类数 $\in[0,39]$ 、推理 "去重类数=38-39" 质检;
- **结果归因 + 记忆沉淀**:每个数字变化 = 改了一个旋钮;事实/教训写进 `memory` 和 `doc/` (PLAN/EXECUTION_STEPS/TODO)。

③ 项目实例

- `test_submission.sh`: `CUDA_VISIBLE_DEVICES="" bash ...` 在登录节点 CPU 推理(修复 CPU 节点 torch 误判 cuda 的 no devices 坑);
- `quantize_pack.py` 内置"总大小 <100MB"检查,超了给三条退路;
- `tests/selftest_*.py`:双分支、auxpose-thermal 等关键模块上服务器前先用合成数据自测。

④ 适用边界

- 过度工程也会拖慢节奏——自测只给"高风险/易错"模块(跨相机对齐、父类委托);
- 环境的可迁移(Torch/CUDA/ultralytics 版本)要写进 `requirements` 复现。

⑤ 面试一句话

"工程上我坚持'实验即代码 + 一键 SBATCH + 防御性断言 + 结果归因 + 记忆沉淀',并攻克了 A100 锁资源、HF 镜像下载、CRLF 换行、CPU 推理误判等环境坑——竞赛能打到多高,一半是这套工程纪律撑起来的。"

M38. 小数据深度学习的完整策略(总纲收尾)

① 思想

当你的数据只有几千、模型有约束、任务要泛化时,找增益要**按性价比排序**,不要乱打拳。本项目把这套顺序验证到位:

② 具体实现(本项目完整路径)

- ① 预训练迁移(第一桶金): IG-65M(R2+1D-34) / MotionBERT(骨架)
→ 视觉 +10~17%, 骨架 +7%
- ② 数据增强(免费正则): aug_strength 分档 + per-clip 一致 → +4.7pt
- ③ 正则与校准: label smoothing / dropout / 早停 / 类平衡
- ④ 数据量(大杠杆): 全量 18 被试 +45% + 多 seed → 0.706→0.711
- ⑤ 模型结构(边际最小): 换结构前先查 lr/归一化/表示/评估协议
- ⑥ 独立视角(最后): thermal 独立相机 → 决策级 +1.86pt → 0.73

③ 项目实例

把这条总纲在面试里讲出来的顺序,就是"先数据、再配方、再结构、再融合"——每一项都有数字背书(0.36 → 0.46 → 0.50 → 0.60 → 0.71 → 0.73),比"我用了 R2+1D、YOLO、量化"有说服力一百倍。

④ 适用边界

- 数据量是硬边界(2891 同域样本):到 0.73 后,再往上更依赖"信息独立性高的新视角"而非"更好的 CNN";
- 每条经验要标注适用条件(哪些在 fold 成立、哪些在全量成立、哪些在 transformer 成立)。

⑤ 面试一句话

"我的小数据方法论总纲是'预训练 > 增强 > 正则 > 结构 > 数据'——每步都有消融背书:预训练 +10~17%、增强 +4.7pt、全量 +45% 数据、独立视角最后 +1.86pt 到 0.73。这套按性价比排序的实验逻辑,比记一堆技巧更有迁移价值。"

结尾:一张纸复述整个项目(面试前背诵版)

1. 任务: RGB-Free / 跨被试(测试全新 12 人)/ ≤100MB, 40 类日常动作, CUHK-X 小模型赛道
2. 硬件拓扑 → 对齐: NYX650 同出 Depth+IR+Skeleton(帧对齐→早融合);
Thermal 独立相机(只能晚融合); Skeleton 3D 米制(只能独立建模)
3. 主线: Depth+IR 4ch → YOLO(IR 上)固定窗口裁剪(保位移)
→ R2+1D-34 + IG-65M(stem 4ch, lr=1e-4)
→ 类平衡 + aug2 + label smoothing + 全量 18 被试 + dual seed
→ 0.71144 LB
4. Thermal: 独立视角 R2+1D-34 + Kinetics 归一化 + 全套 recipe → int5 40MB
5. 融合: prob 平均 + flip TTA = 0.73 LB(80MB, 1×main + 1×thermal 独立视角 > 2×main)
6. 量化: per-channel int5 bit-packing, 只量化大权重, int4 崩 → 铁律最低 int5
7. 关闭清单(要能讲清每个为什么):
subject 泄漏(→ 重建协议) | lr 冲垮预训练(→1e-4) | Cosine 归零(→StepLR, transformer 除外)
SWA 塌缩(→prob 平均) | int4 崩 | 对比/伪标签/门控/特征融合/auxpose/IMU/NTU/时间 TTA...全 fold0 判据关闭
8. 三条宪法: 运动优先 / 检测定位裁剪保运动 / 单模态达标再集成
9. 实验纪律: fold0 判据 / 不烧 LB / 保底不动 / oracle 先算天花板 / 结果归因到单一旋钮

最后的最后:能让面试官相信"这是你的项目"的,不是这些方法的名字,而是——你知道**每一步为什么这么做、为什么没做另一件事、以及失败时你看到了什么信号做出了什么判断**。上面每一条都有"实例"和"边界",把这两

栏反复过三遍,你就拥有了能对答如流的资本。